

Making the Linux Kernel

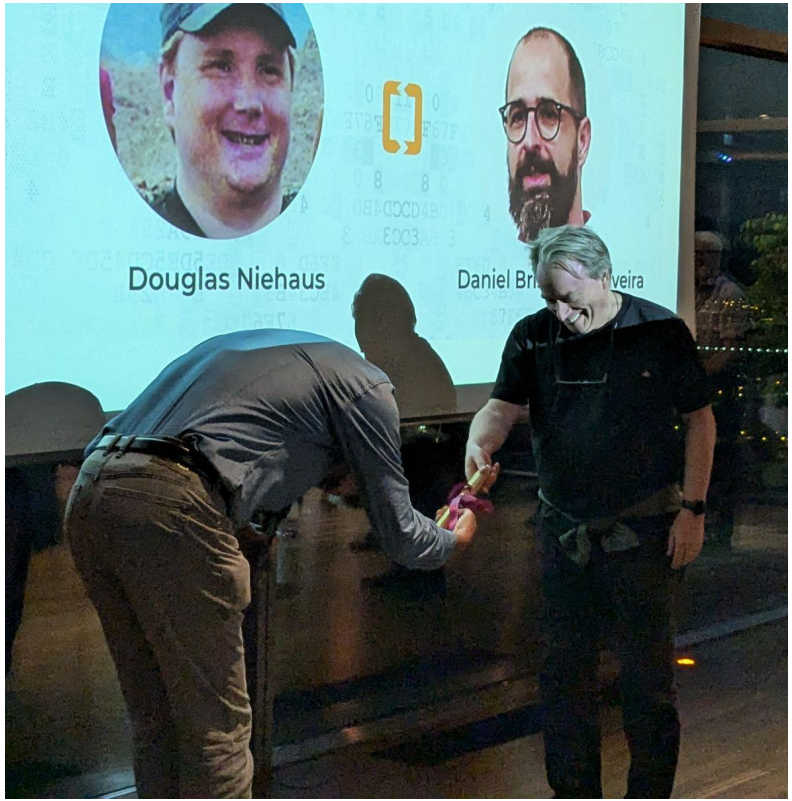


Making the Linux Kernel

SUCK LESS

September 19, 2024





Making the Linux Kernel

SUCK LESS

PREEMPT_RT

Making the Linux Kernel

SUCK LESS

In memory of Daniel Bristot de Oliviera



Real-Time Going Upstream 6.12!!!

- In honor of RT going upstream, need to list what it did for the kernel
  - `mutex_lock`
  - NO HZ
  - High Resolution Timers
  - lockdep
  - Priority Inheritance (futex)
  - Threaded interrupts
  - ftrace
  - printk

mutex_lock/unlock

- All sleeping locks use to be just a semaphore

mutex_lock/unlock

- All sleeping locks use to be just a semaphore
  - semaphores allow a set of tasks to enter a critical section

mutex_lock/unlock

- All sleeping locks use to be just a semaphore
  - semaphores allow a set of tasks to enter a critical section
  - Most semaphores only allow just one task (mutex!)

mutex_lock/unlock

- All sleeping locks use to be just a semaphore
  - semaphores allow a set of tasks to enter a critical section
  - Most semaphores only allow just one task (mutex!)
  - Semaphores allowed passing of ownership

mutex_lock/unlock

- All sleeping locks use to be just a semaphore
  - semaphores allow a set of tasks to enter a critical section
  - Most semaphores only allow just one task (mutex!)
  - Semaphores allowed passing of ownership
- RT requires an “owner” for all critical sections
  - This is a requirement for priority inheritance

mutex_lock/unlock

- All sleeping locks use to be just a semaphore
  - semaphores allow a set of tasks to enter a critical section
  - Most semaphores only allow just one task (mutex!)
  - Semaphores allowed passing of ownership
- RT requires an “owner” for all critical sections
  - This is a requirement for priority inheritance
- The RT folks added the `mutex_lock/unlock` interface

NO_HZ and High Resolution Timers

- The tick use to always run
  - Even when the system was idle

NO_HZ and High Resolution Timers

- The tick use to always run
  - Even when the system was idle
  - Prevents the CPU from entering a deep sleep
    - Expensive for large data centers

NO_HZ and High Resolution Timers

- The tick use to always run
  - Even when the system was idle
  - Prevents the CPU from entering a deep sleep
    - Expensive for large data centers
- High Resolution Timers was being blocked by the timer maintainer

NO_HZ and High Resolution Timers

- The tick use to always run
  - Even when the system was idle
  - Prevents the CPU from entering a deep sleep
    - Expensive for large data centers
- High Resolution Timers was being blocked by the timer maintainer
  - NO_HZ was made dependent on HR Timers
  - Couldn’t have NO_HZ without HR Timers

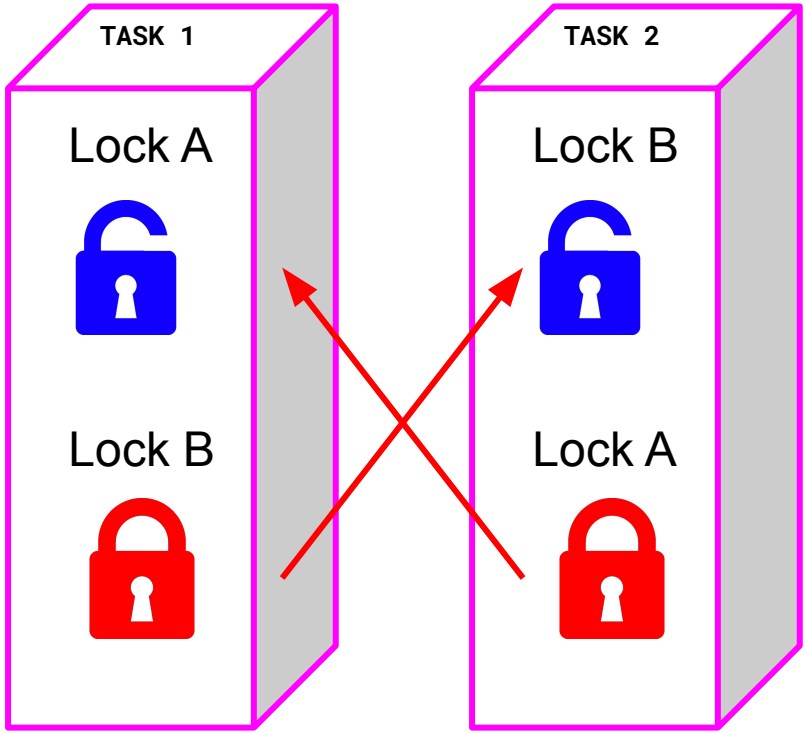
NO_HZ and High Resolution Timers

- The tick use to always run
  - Even when the system was idle
  - Prevents the CPU from entering a deep sleep
    - Expensive for large data centers
- High Resolution Timers was being blocked by the timer maintainer
  - NO_HZ was made dependent on HR Timers
  - Couldn’t have NO_HZ without HR Timers
  - Linus pulled in both

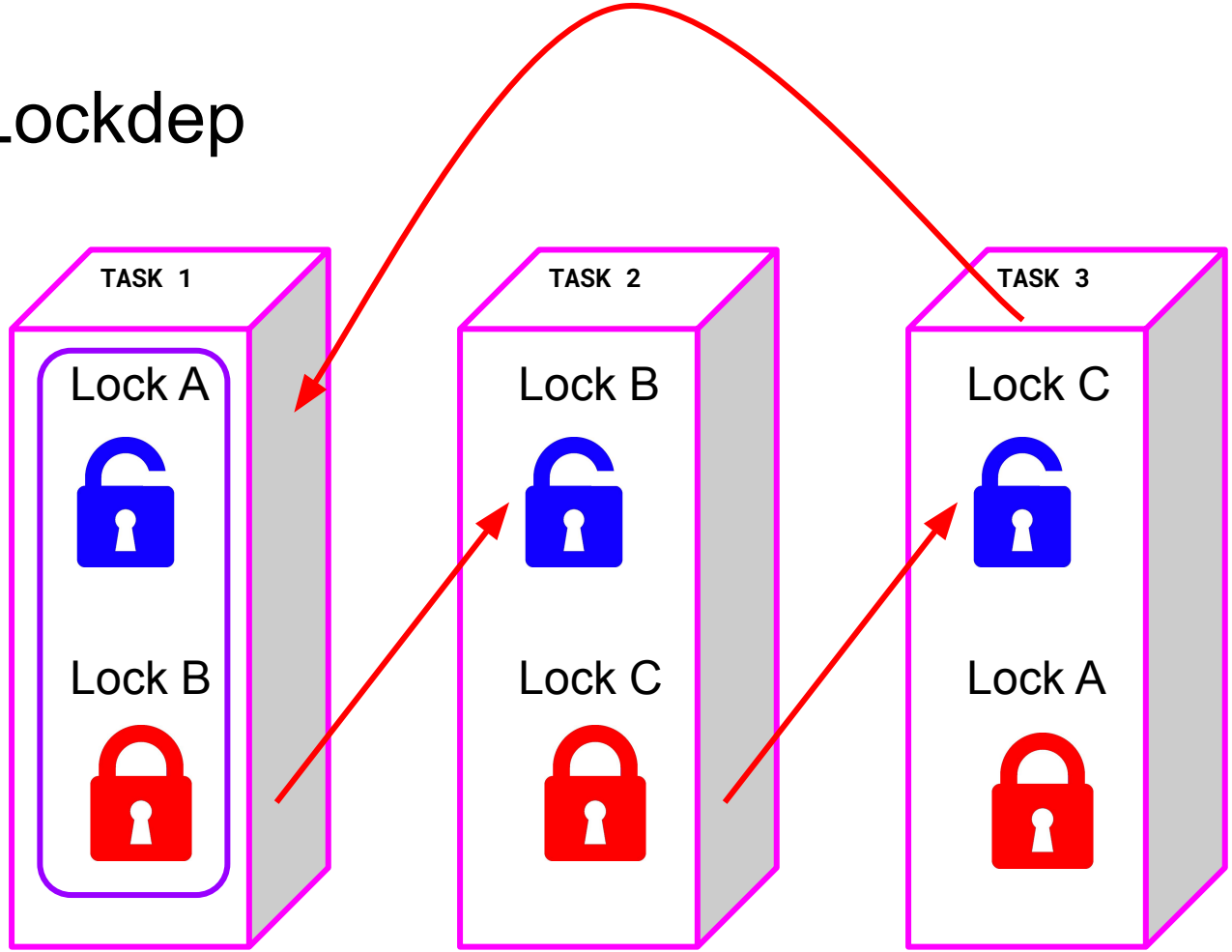
NO_HZ and High Resolution Timers

- The tick use to always run
  - Even when the system was idle
  - Prevents the CPU from entering a deep sleep
    - Expensive for large data centers
- High Resolution Timers was being blocked by the timer maintainer
  - NO_HZ was made dependent on HR Timers
  - Couldn’t have NO_HZ without HR Timers
  - Linus pulled in both
  - Thomas Gleixner became the new timer maintainer

Lockdep



Lockdep



Lockdep

Lock A



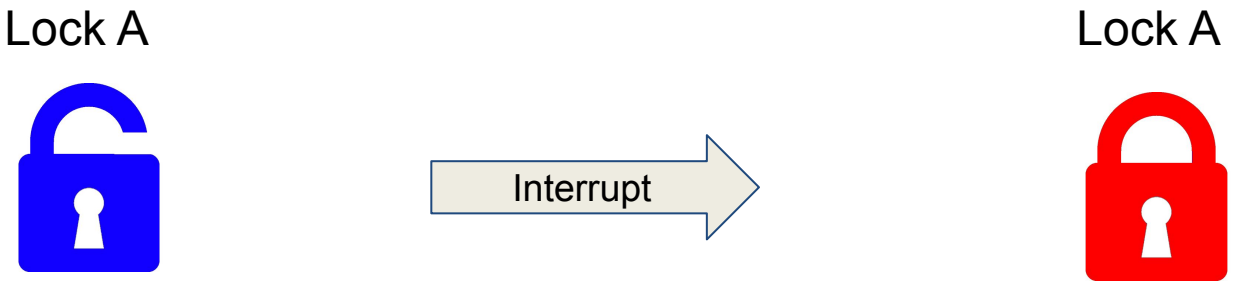
Lockdep

Lock A

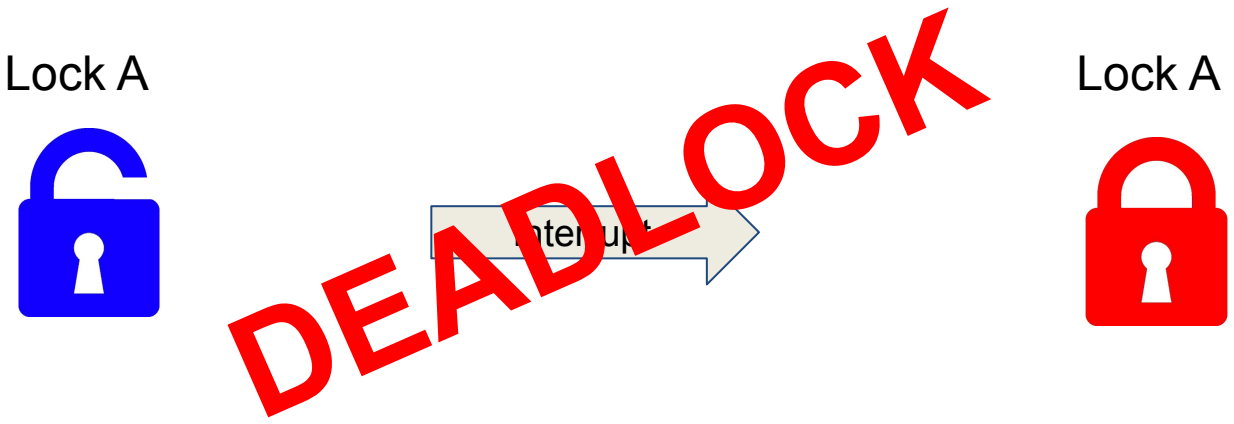




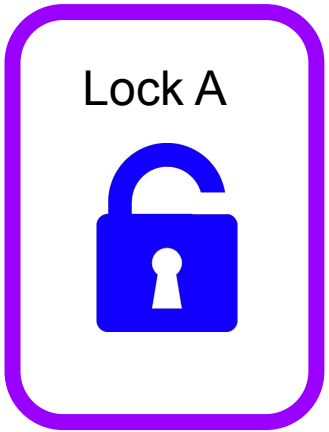
Lockdep



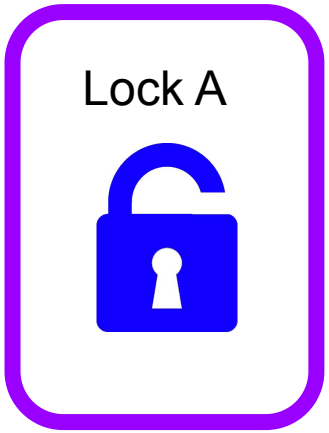
Lockdep

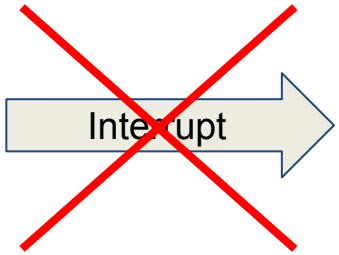


Lockdep

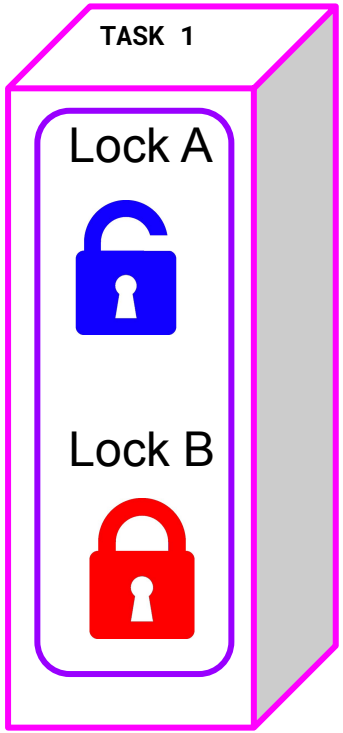


Lockdep

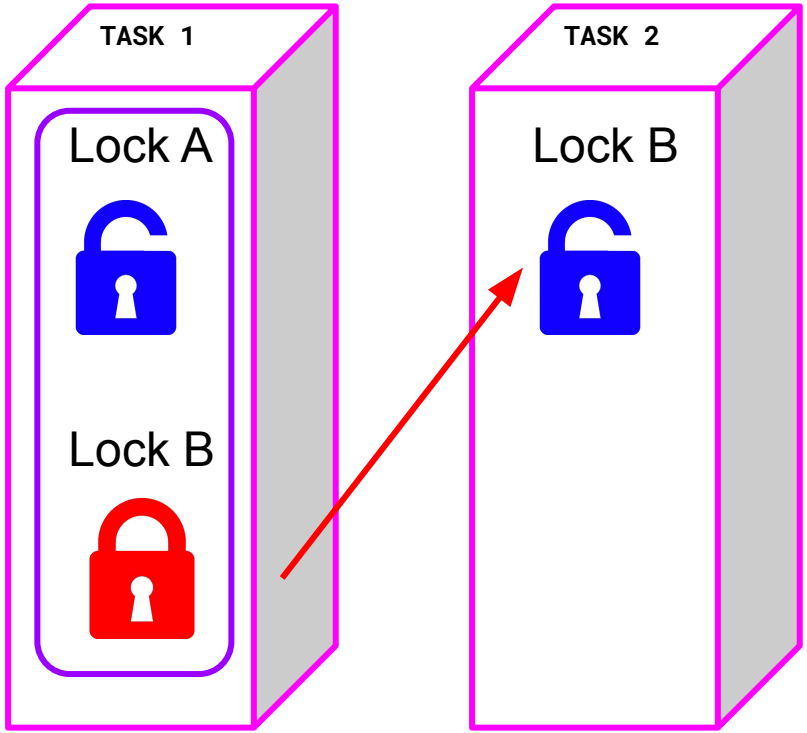




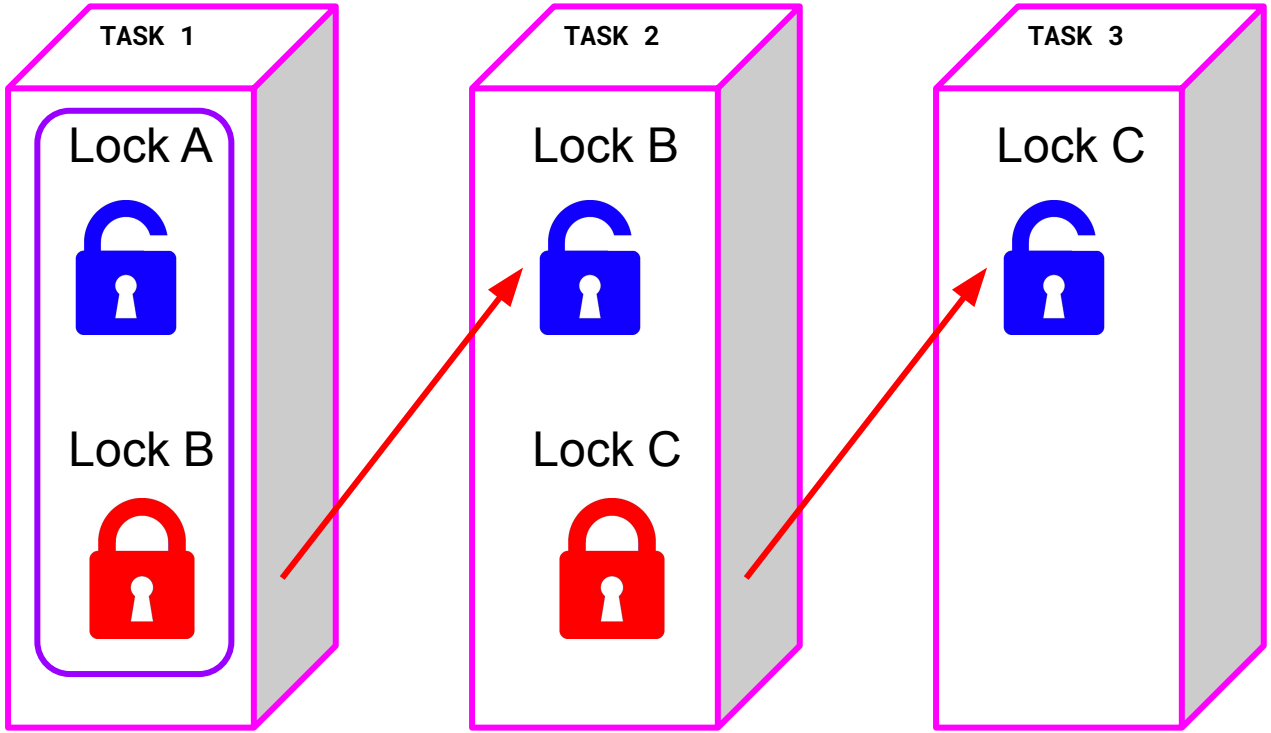
Lockdep



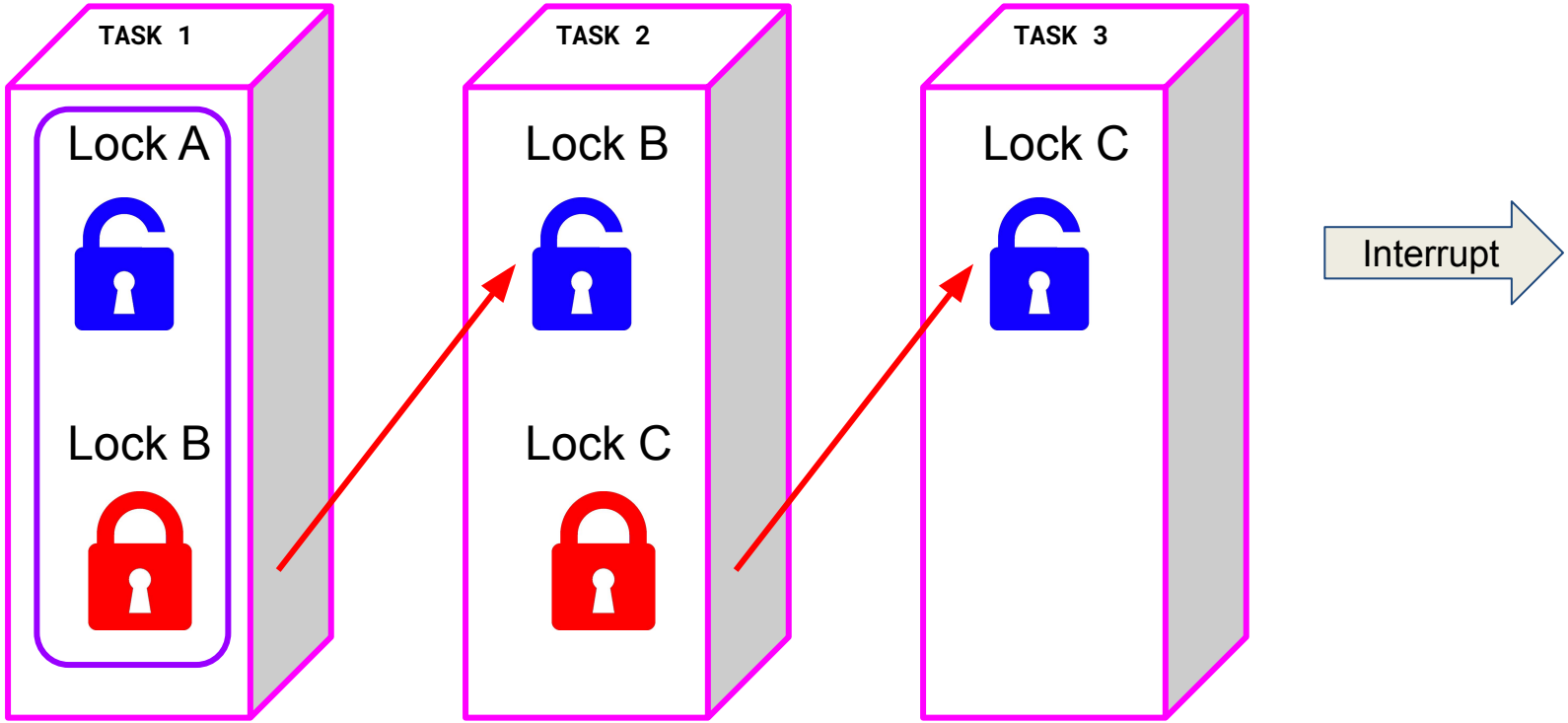
Lockdep



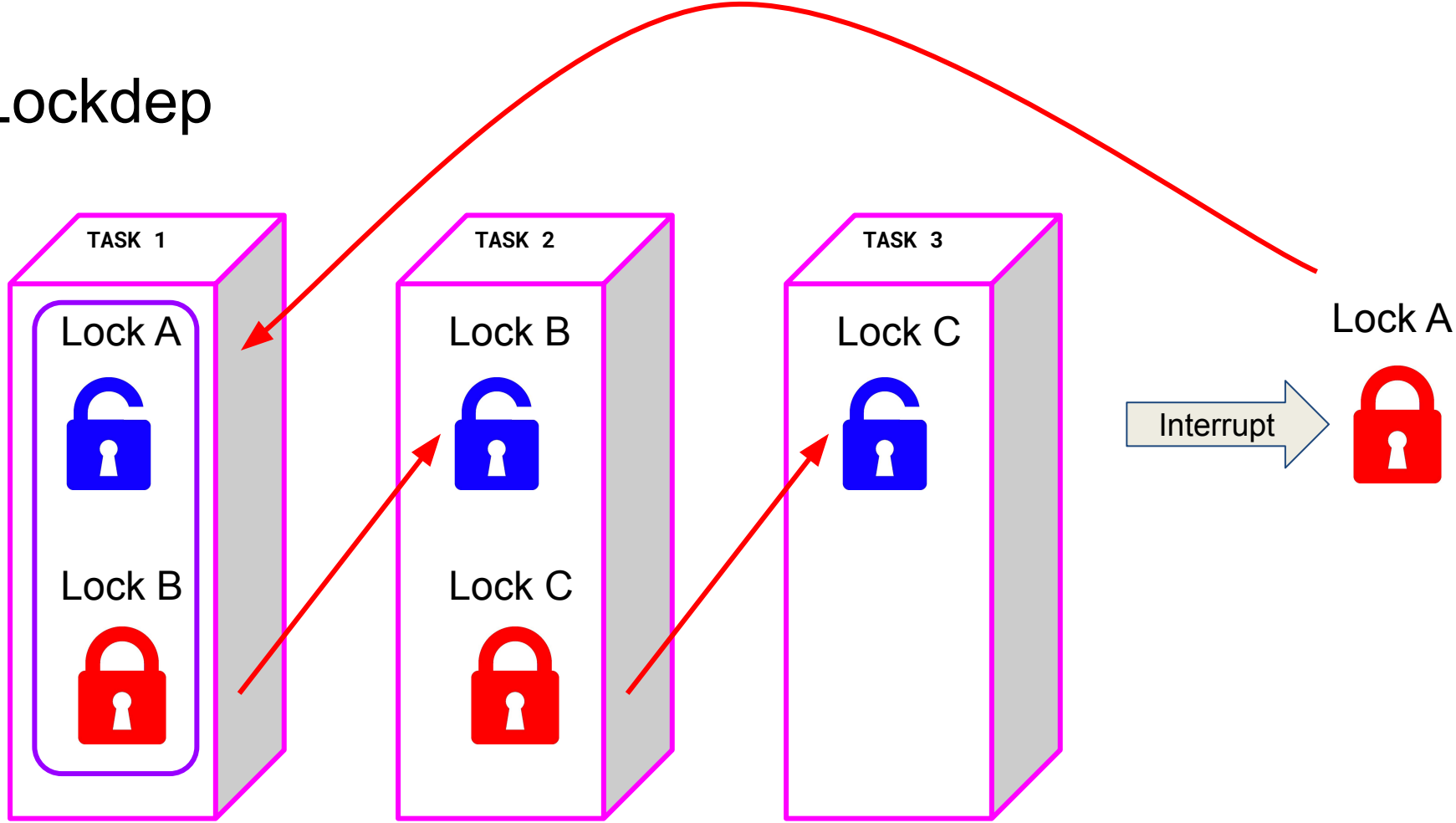
Lockdep



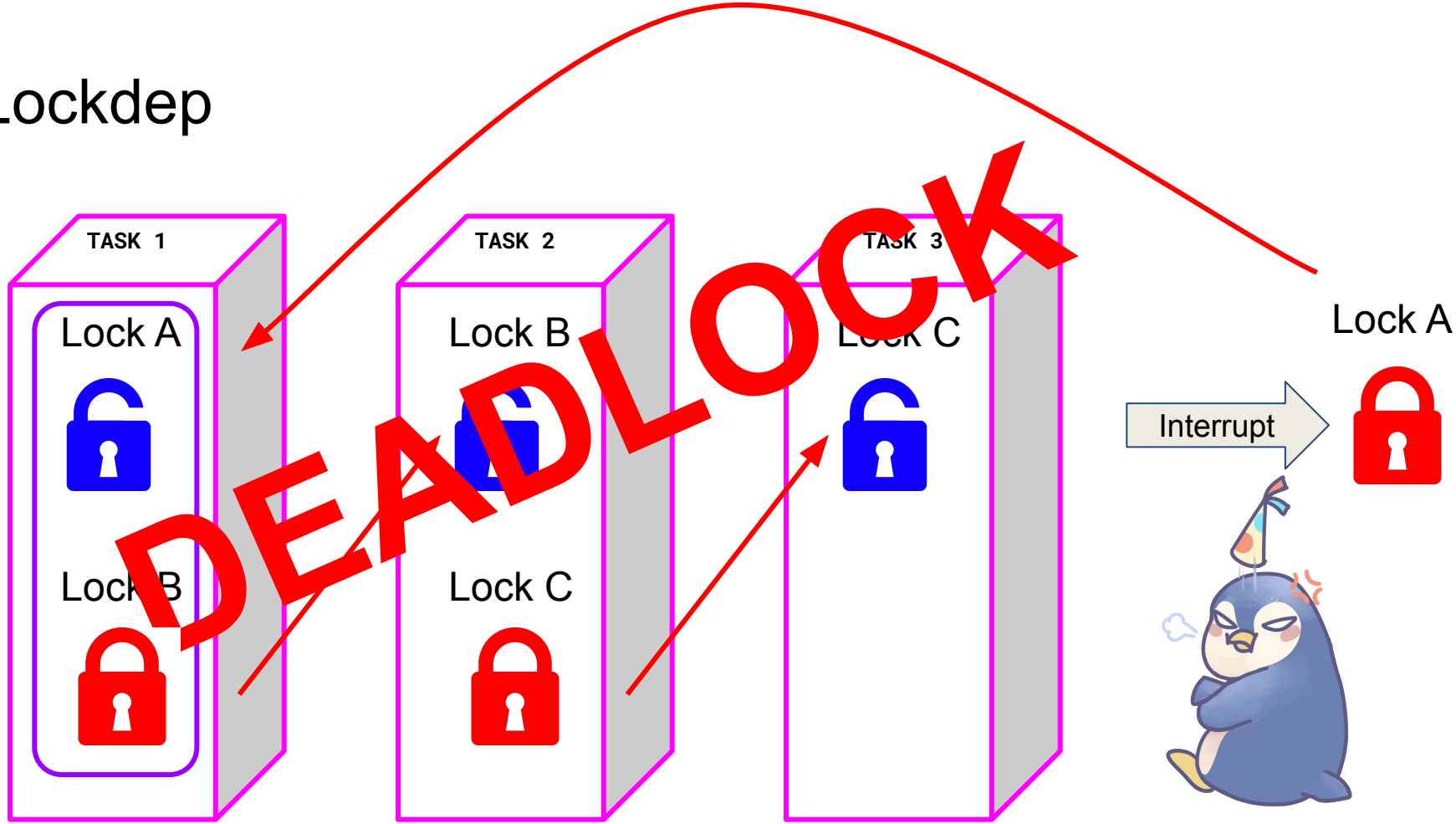
Lockdep



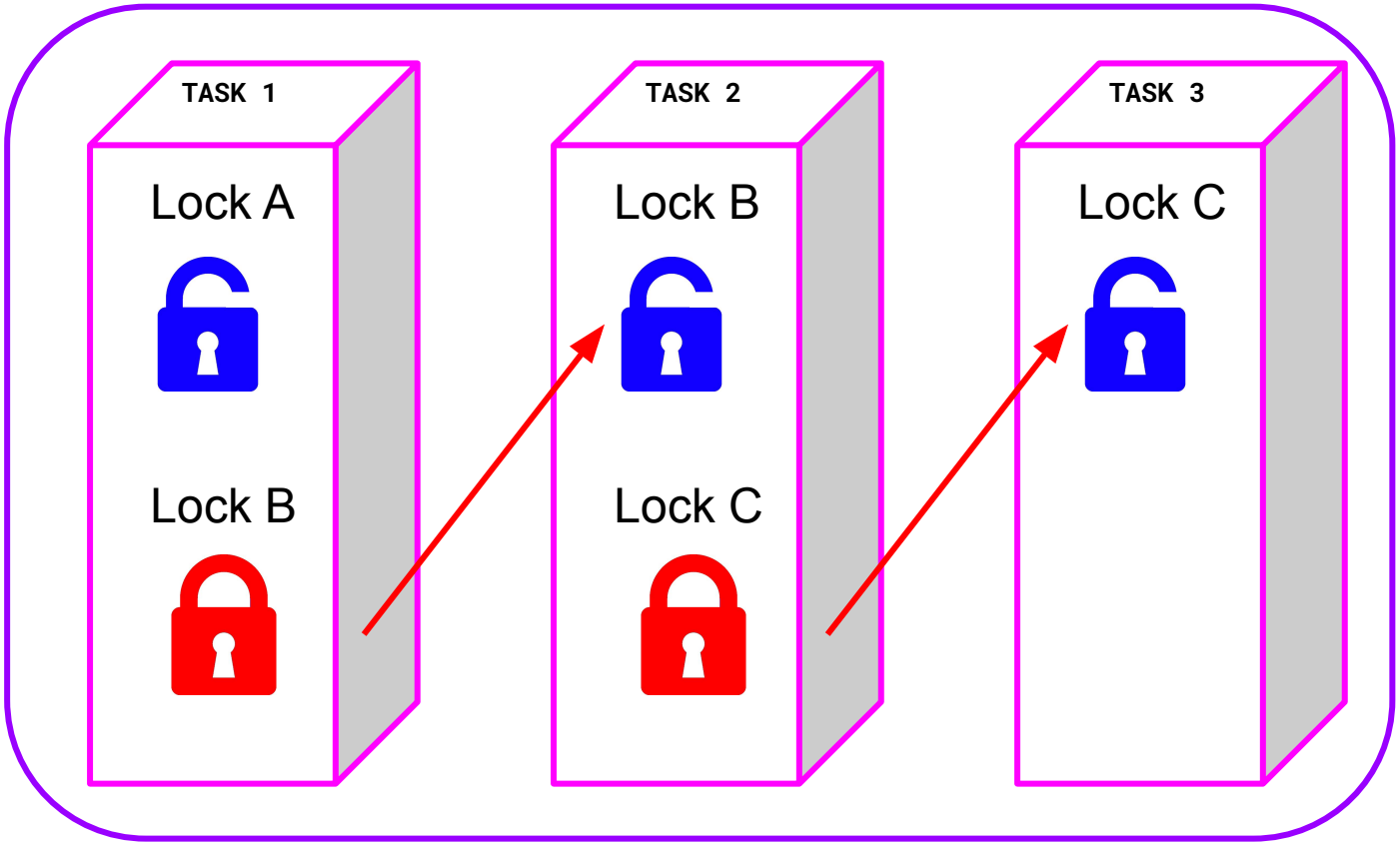
Lockdep

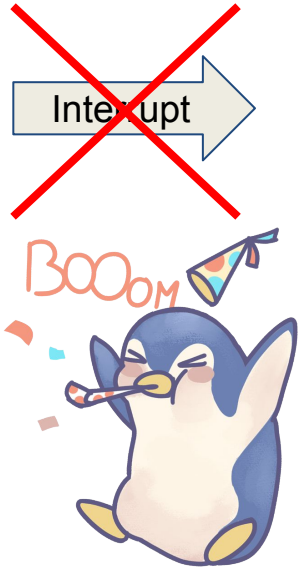

TASK 1
Lock A
Lock B
TASK 2
Lock B
Lock C
TASK 3
Lock C
Lock A
Interrupt

Lockdep


TASK 1
Lock A
Lock B
TASK 2
Lock B
Lock C
TASK 3
Lock C
DEADLOCK
Lock A
Interrupt

Lockdep

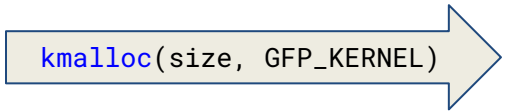

TASK 1
Lock A
Lock B
TASK 2
Lock B
Lock C
TASK 3
Lock C


Interrupt
BOOOM

Lockdep

Lock A

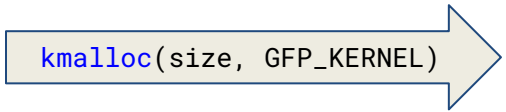


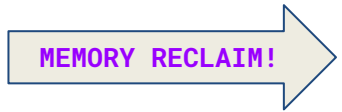


Lockdep

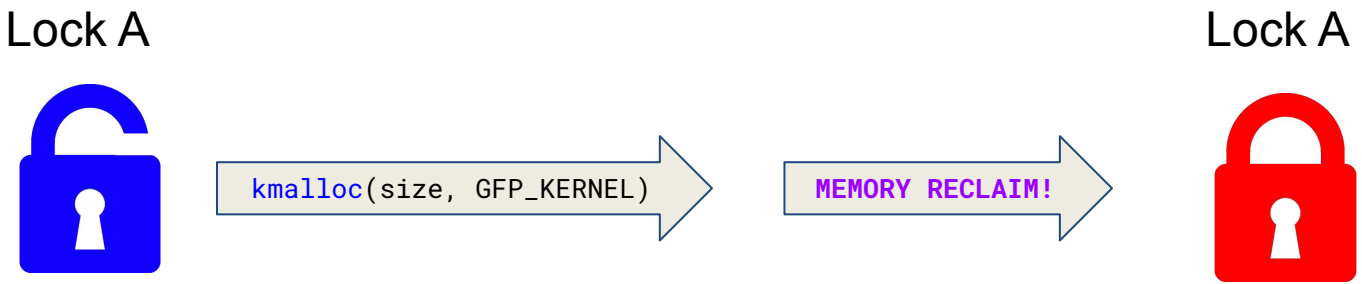
Lock A



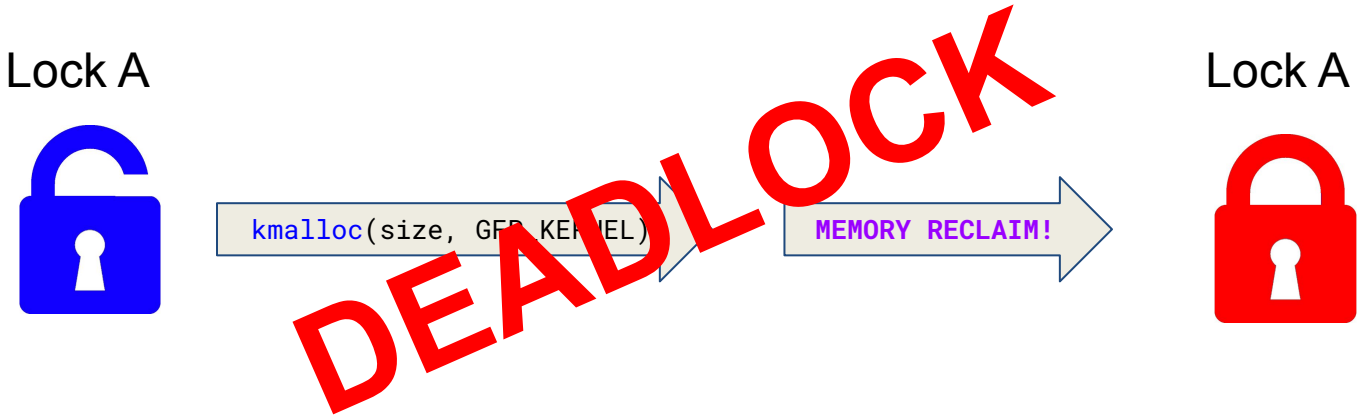




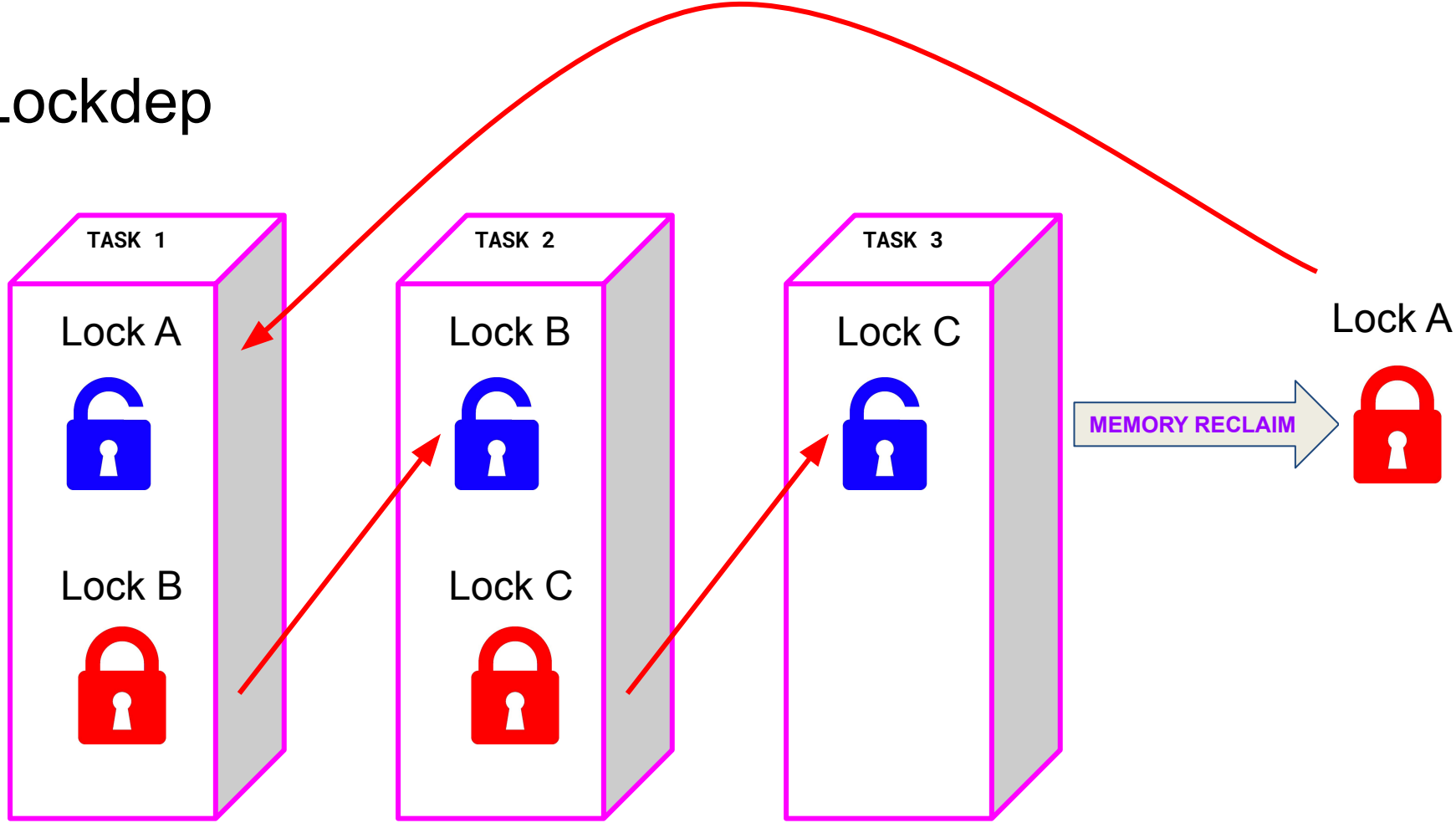
Lockdep



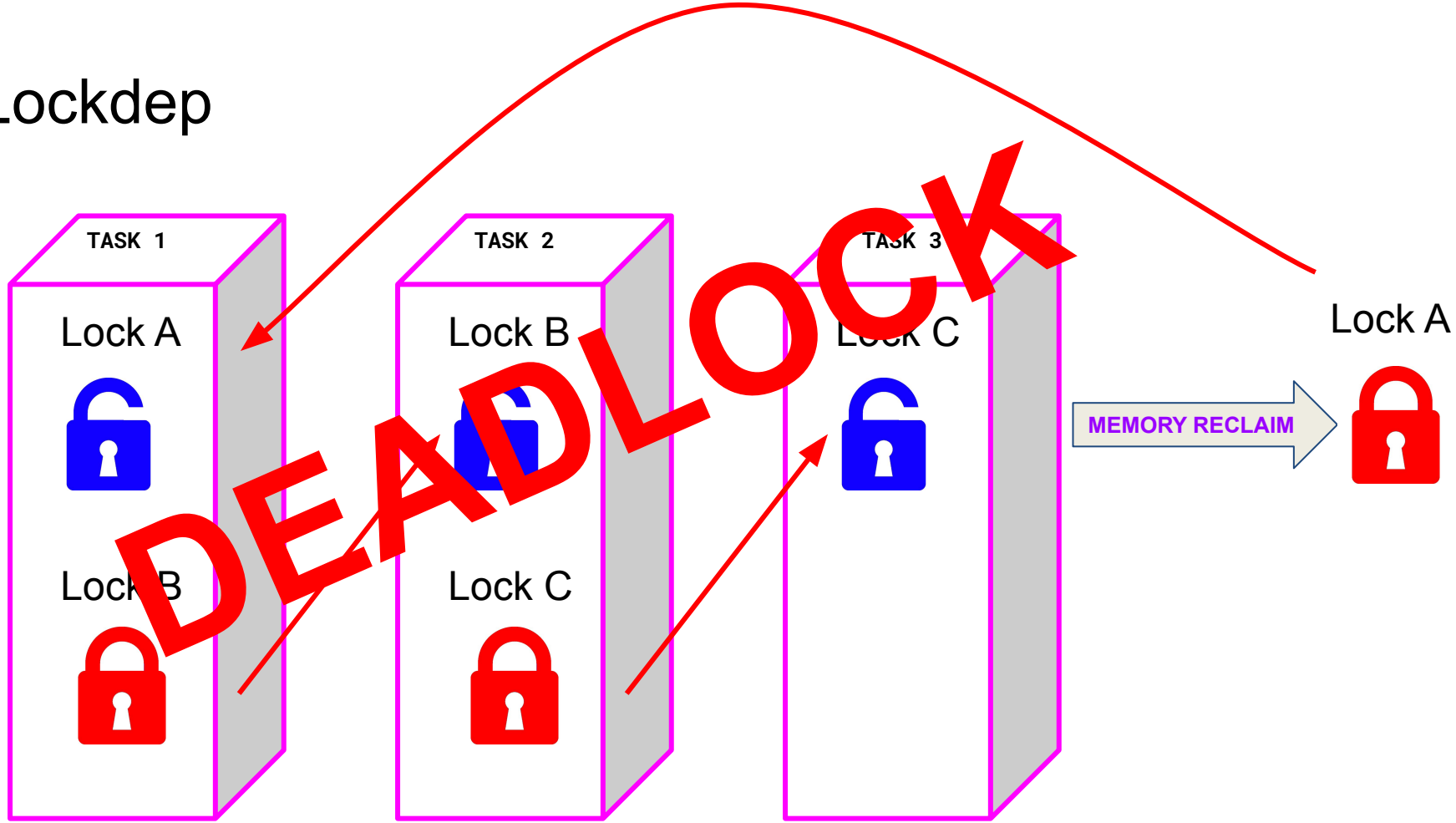
Lockdep


Lock A
kmalloc(size, GFP_KERNEL)
MEMORY RECLAIM!
Lock A
DEADLOCK

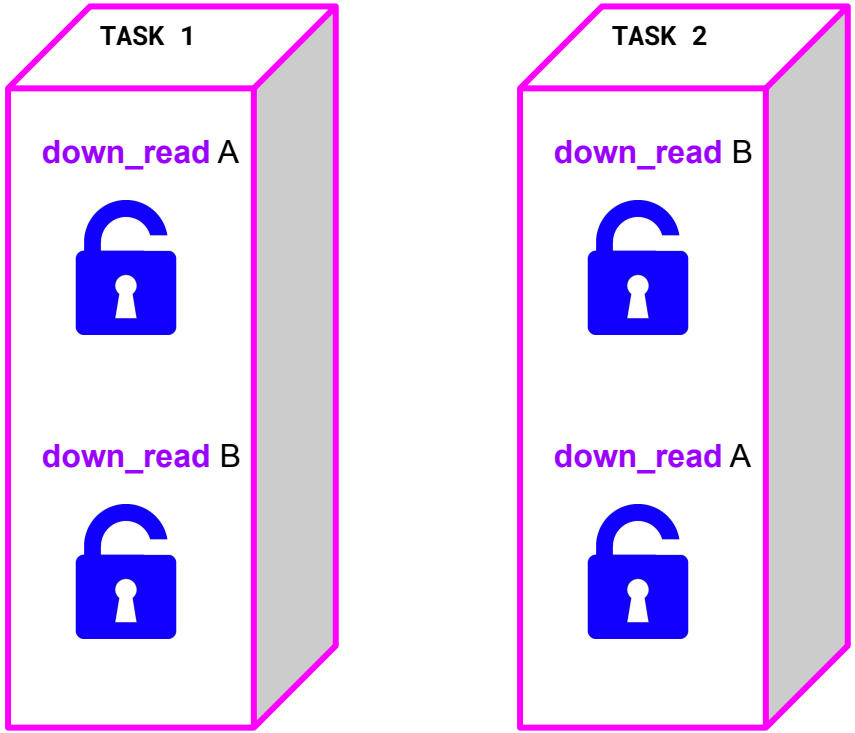
Lockdep


TASK 1
Lock A
Lock B
TASK 2
Lock B
Lock C
TASK 3
Lock C
MEMORY RECLAIM
Lock A

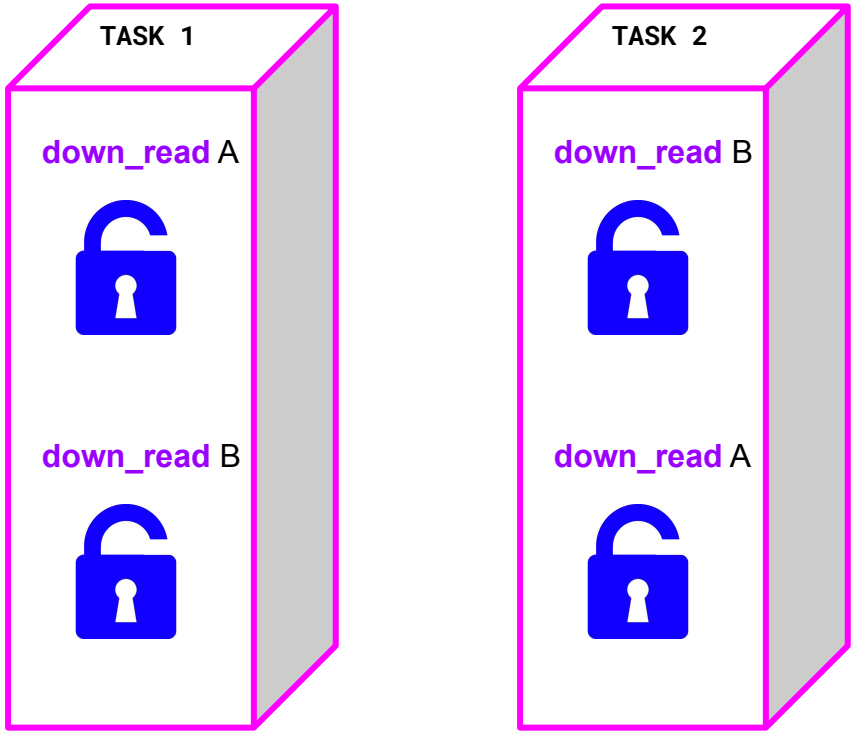
Lockdep


TASK 1
Lock A
Lock B
TASK 2
Lock B
Lock C
TASK 3
Lock C
DEADLOCK
MEMORY RECLAIM
Lock A

Lockdep

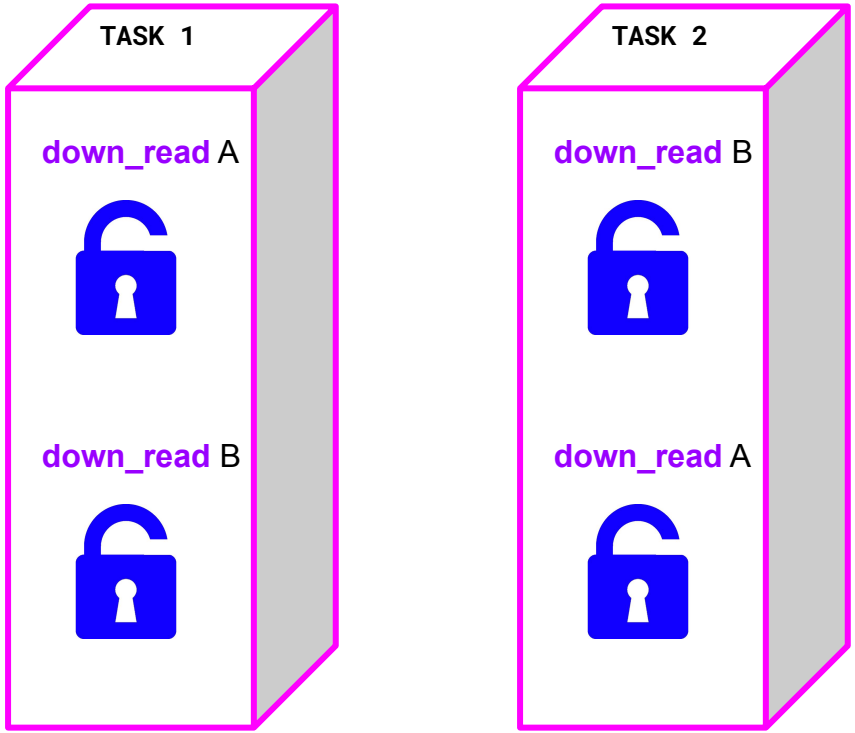


Lockdep



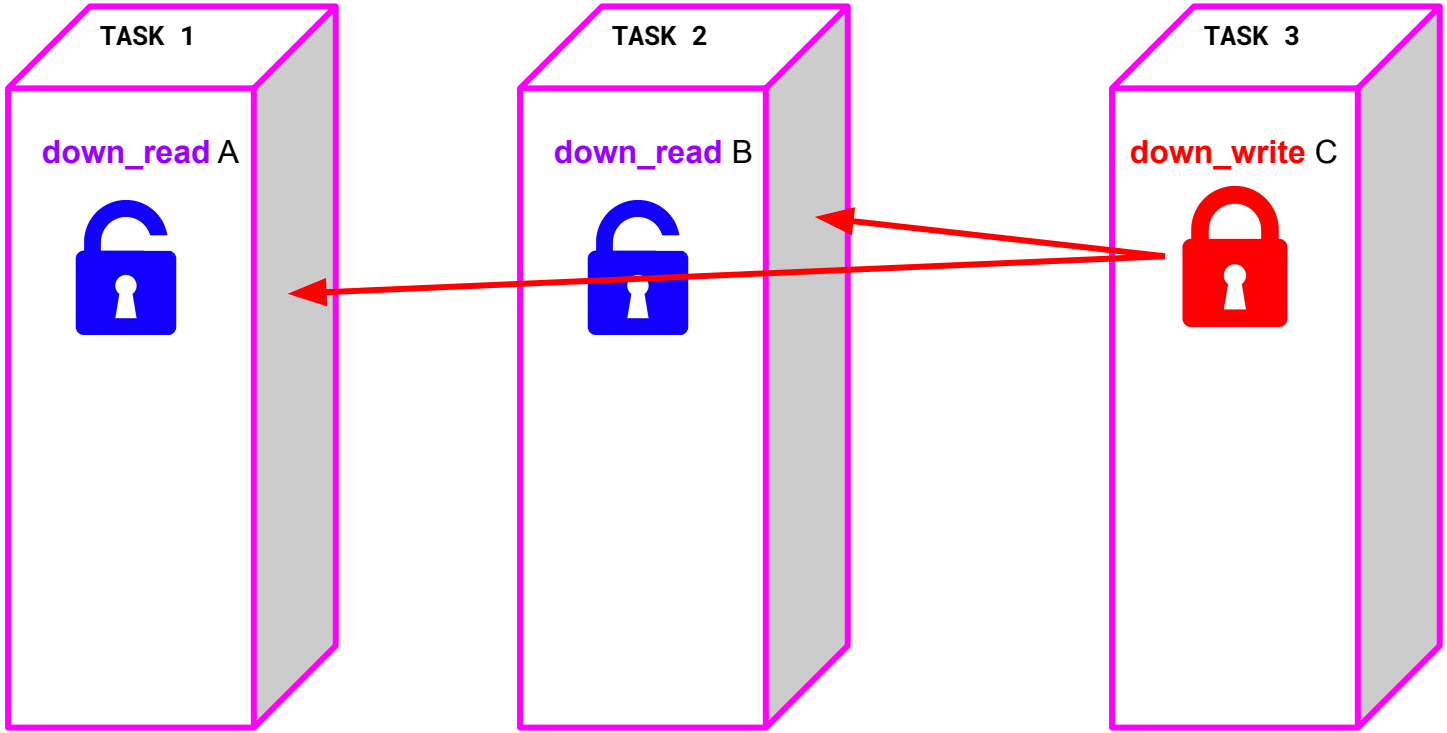
DEADLOCK??

Lockdep

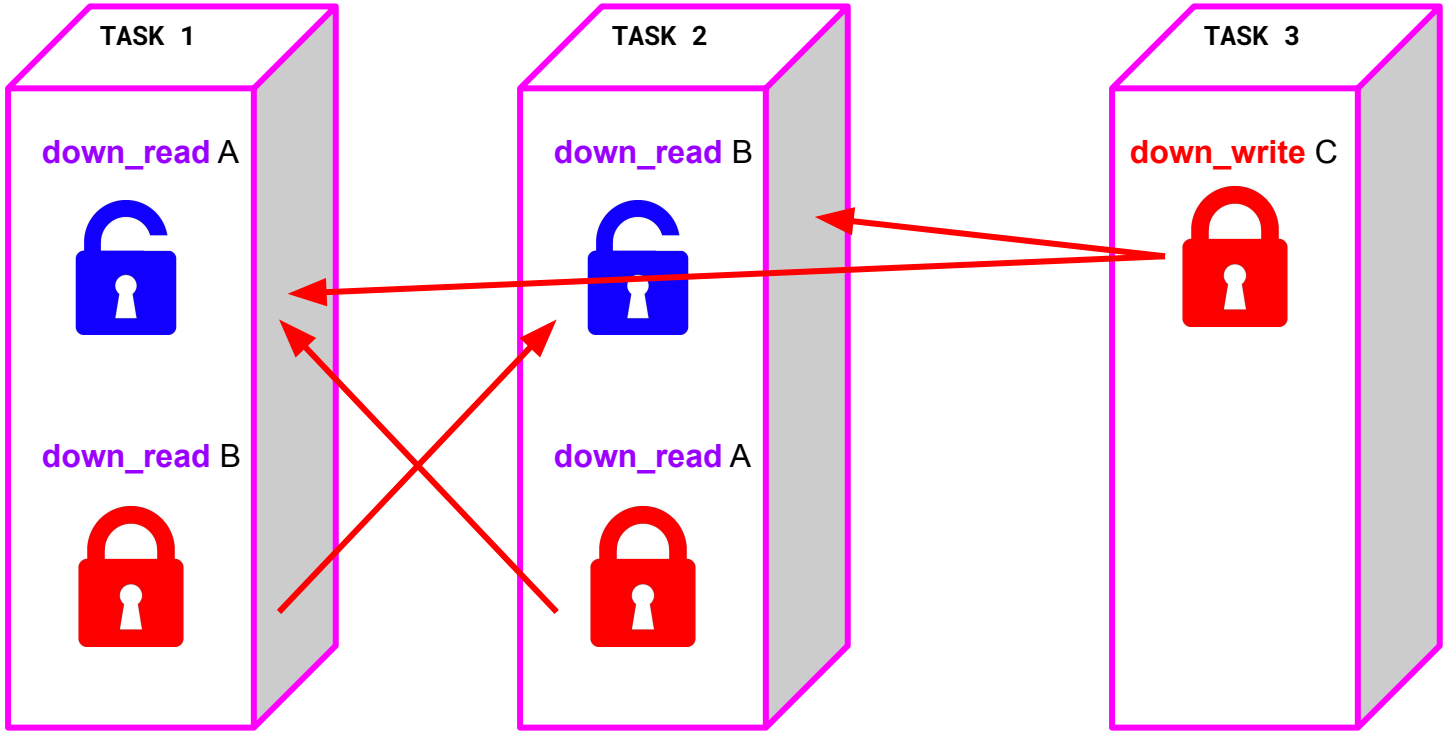

TASK 1
down_read A
down_read B
TASK 2
down_read B
down_read A


DEADLOCK??
YES

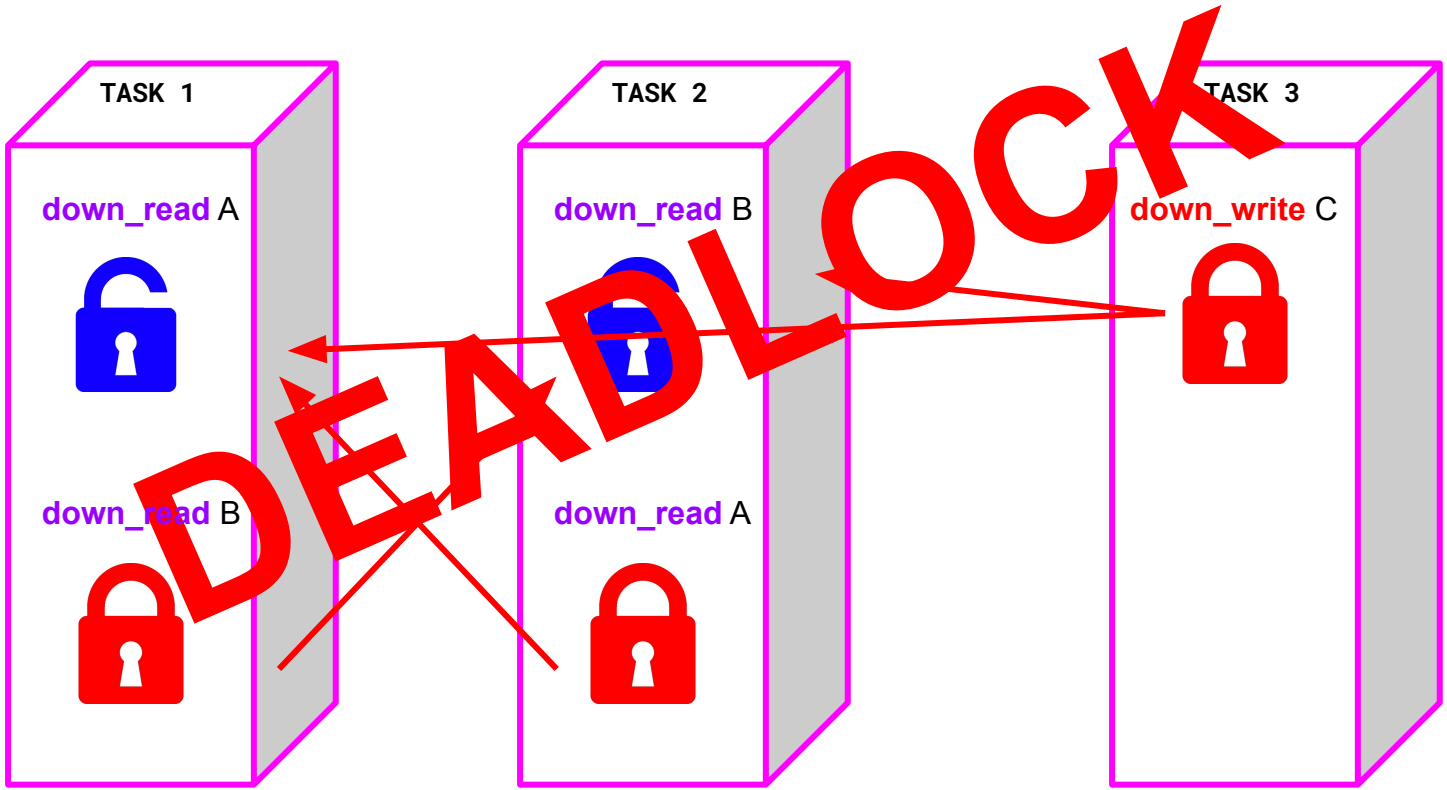
Lockdep



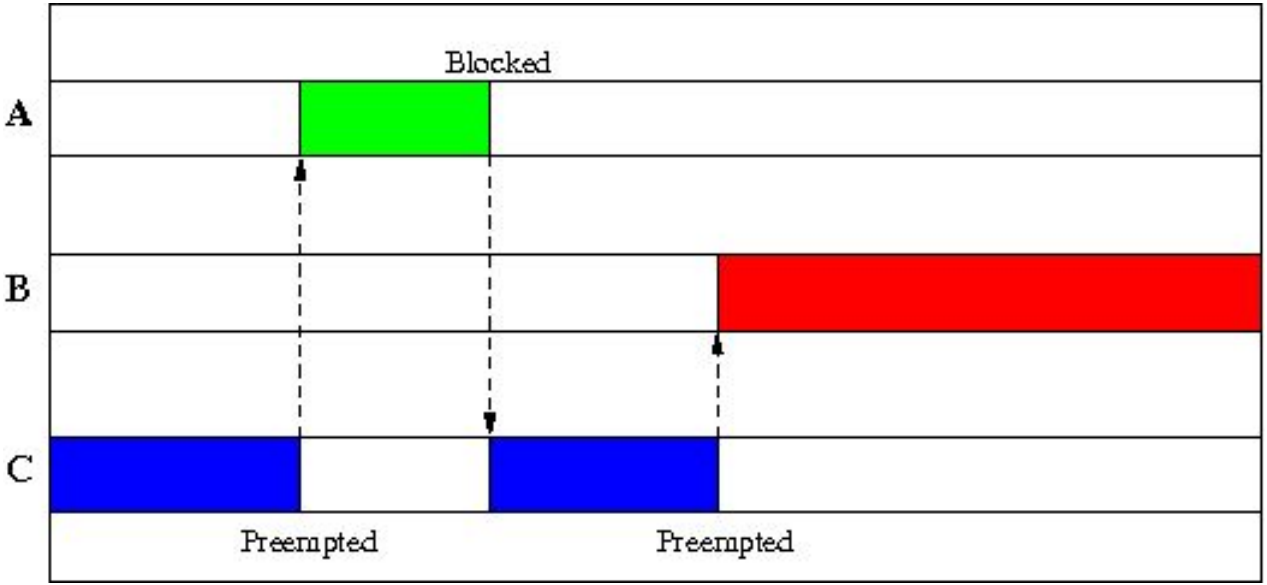
Lockdep


TASK 1
down_read A
down_read B
TASK 2
down_read B
down_read A
TASK 3
down_write C

Lockdep

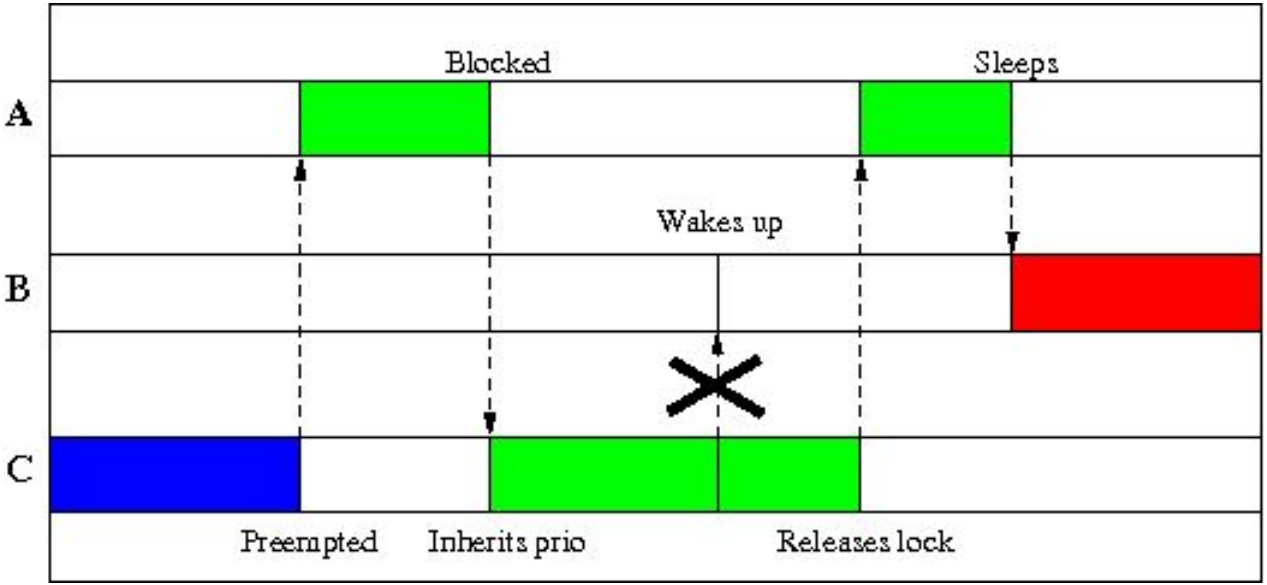

TASK 1
down_read A
down_read B
TASK 2
down_read B
down_read A
TASK 3
down_write C
DEADLOCK

User Space Priority Inheritance

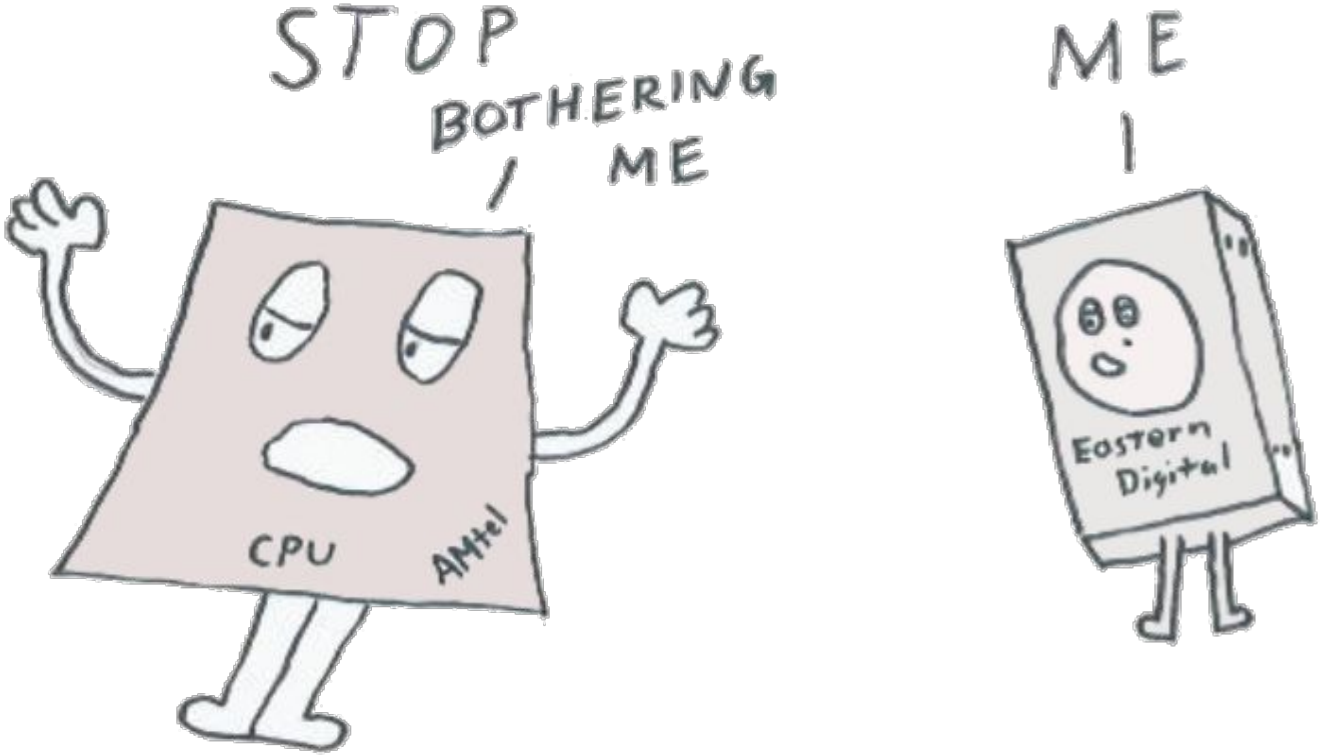


User Space Priority Inheritance

```
pthread_mutexattr_setprotocol(&attr, PTHREAD_PRIO_INHERIT)
```

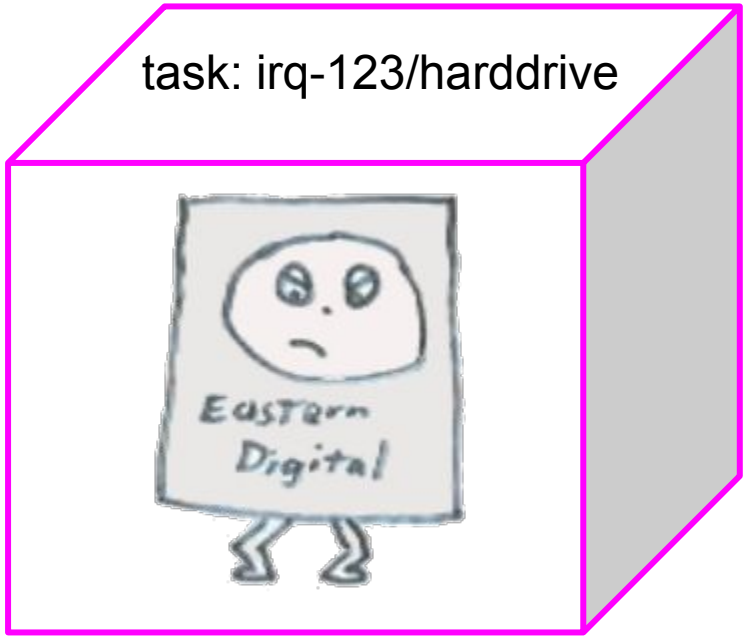


Interrupts



Interrupt threads





Tracing / ftrace

- Three month project
  - Started in January 2008
  - Was to port the tracing infrastructure from PREEMPT_RT

Tracing / ftrace

- Three month project
  - Started in January 2008
  - Was to port the tracing infrastructure from PREEMPT_RT
- Still haven’t finished it



printk

```
# cyclictest -q -D 90s -i 500 -d 500
T: 0 ( 3634) P: 0 I:500 C: 180000 Min:     10 Act:   54 Avg:   54 Max:      68
```

printk

```
# cyclictest -q -D 90s -i 500 -d 500
T: 0 ( 3634) P: 0 I:500 C: 180000 Min:     10 Act:   54 Avg:   54 Max:      68

# cyclictest -q -D 90s -i 500 -d 500
```

printk

```
# cyclictest -q -D 90s -i 500 -d 500
T: 0 ( 3634) P: 0 I:500 C: 180000 Min:     10 Act:   54 Avg:   54 Max:      68

# cyclictest -q -D 90s -i 500 -d 500
```

```
# echo ? > /proc/sysrq-trigger
```

printk

```
# cyclictest -q -D 90s -i 500 -d 500
T: 0 ( 3634) P: 0 I:500 C: 180000 Min:     10 Act:   54 Avg:   54 Max:      68

# cyclictest -q -D 90s -i 500 -d 500
```

```
# echo ? > /proc/sysrq-trigger
```



```
T: 0 ( 3653) P: 0 I:500 C: 179931 Min:      6 Act:   55 Avg:   55 Max:   34658
```

printk

- The last blocker of PREEMPT_RT!

printk

- The last blocker of PREEMPT_RT!
- Old printk serializes the output
  - Does each console one by one!

printk

- The last blocker of PREEMPT_RT!
- Old printk serializes the output
  - Does each console one by one!
- Threaded printk allows all consoles to be printed at once!

printk

- The last blocker of PREEMPT_RT!
- Old printk serializes the output
  - Does each console one by one!
- Threaded printk allows all consoles to be printed at once!
- Can now be called in **any** context!
  - NMI
  - Scheduler

Build speed

- In 2008, ftrace was added

Build speed

- In 2008, ftrace was added
- It originally required a daemon to find the `mcount` locations
  - This is needed to convert them to nops

Build speed

- In 2008, ftrace was added
- It originally required a daemon to find the `mcount` locations
  - This is needed to convert them to nops
- To get rid of the daemon, Perl scripts were added to the build

Build speed

- In 2008, ftrace was added
- It originally required a daemon to find the `mcount` locations
  - This is needed to convert them to nops
- To get rid of the daemon, Perl scripts were added to the build
  - It did `objdump` to find the locations of mcount calls

Build speed

- In 2008, ftrace was added
- It originally required a daemon to find the `mcount` locations
  - This is needed to convert them to nops
- To get rid of the daemon, Perl scripts were added to the build
  - It did `objdump` to find the locations of mcount calls
  - It then created an assembly file holding these locations

Build speed

- In 2008, ftrace was added
- It originally required a daemon to find the `mcount` locations
  - This is needed to convert them to nops
- To get rid of the daemon, Perl scripts were added to the build
  - It did `objdump` to find the locations of mcount calls
  - It then created an assembly file holding these locations
  - It then compiled and relinked the file into the original object file

Build speed

- In 2008, ftrace was added
- It originally required a daemon to find the `mcount` locations
  - This is needed to convert them to nops
- To get rid of the daemon, Perl scripts were added to the build
  - It did `objdump` to find the locations of mcount calls
  - It then created an assembly file holding these locations
  - It then compiled and relinked the file into the original object file
- Needless to say, this slowed down the build

Build speed

- In 2008, ftrace was added
- It originally required a daemon to find the `mcount` locations
  - This is needed to convert them to nops
- To get rid of the daemon, Perl scripts were added to the build
  - It did `objdump` to find the locations of mcount calls
  - It then created an assembly file holding these locations
  - It then compiled and relinked the file into the original object file
- Needless to say, this slowed down the build
- In 2010, `recordmcount.c` was written to do this in C
  - Written by John Reiser
  - sped up the build again!

Build speed

- In 2008, Linus complained about build speed

Build speed

- In 2008, Linus complained about build speed
  - We ask users to bisect the bugs they report

Build speed

- In 2008, Linus complained about build speed
  - We ask users to bisect the bugs they report
  - They only have the distro configs

Build speed

- In 2008, Linus complained about build speed
  - We ask users to bisect the bugs they report
  - They only have the distro configs
  - It can take them 13 hours to build on their machines

Build speed

- In 2008, Linus complained about build speed
  - We ask users to bisect the bugs they report
  - They only have the distro configs
  - It can take them 13 hours to build on their machines
- Linus told Kernel Summit attendees to fix it

Build speed

- In 2008, Linus complained about build speed
  - We ask users to bisect the bugs they report
  - They only have the distro configs
  - It can take them 13 hours to build on their machines
- Linus told Kernel Summit attendees to fix it

(I wasn't invited)

Build speed

- In 2008, Linus complained about build speed
  - We ask users to bisect the bugs they report
  - They only have the distro configs
  - It can take them 13 hours to build on their machines
- Linus told Kernel Summit attendees to fix it
  - Thomas Gleixner said we already have a script

Build speed

- In 2008, Linus complained about build speed
  - We ask users to bisect the bugs they report
  - They only have the distro configs
  - It can take them 13 hours to build on their machines
- Linus told Kernel Summit attendees to fix it
  - Thomas Gleixner said we already have a script
  - “Steven Rostedt has this `streamline-config.pl`”

Build speed

- In 2008, Linus complained about build speed
  - We ask users to bisect the bugs they report
  - They only have the distro configs
  - It can take them 13 hours to build on their machines
- Linus told Kernel Summit attendees to fix it
  - Thomas Gleixner said we already have a script
  - “Steven Rostedt has this `streamline-config.pl`”
  - Linus asked him why it’s not already in the kernel?

Build speed

- In 2008, Linus complained about build speed
  - We ask users to bisect the bugs they report
  - They only have the distro configs
  - It can take them 13 hours to build on their machines
- Linus told Kernel Summit attendees to fix it
  - Thomas Gleixner said we already have a script
  - “Steven Rostedt has this `streamline-config.pl`”
  - Linus asked him why it’s not already in the kernel?
- Later Steven was asked to add it

Build speed

- In 2008, Linus complained about build speed
  - We ask users to bisect the bugs they report
  - They only have the distro configs
  - It can take them 13 hours to build on their machines
- Linus told Kernel Summit attendees to fix it
  - Thomas Gleixner said we already have a script
  - “Steven Rostedt has this `streamline-config.pl`”
  - Linus asked him why it’s not already in the kernel?
- Later Steven was asked to add it
  - This became `make localmodconfig`
  - It sped up the build tremendously

Build speed

- In 2008, Linus complained about build speed
  - We ask users to bisect the bugs they report
  - They only have the distro configs
  - It can take them 13 hours to build on their machines
- Linus told Kernel Summit attendees to fix it
  - Thomas Gleixner said we already have a script
  - “Steven Rostedt has this `streamline-config.pl`”
  - Linus asked him why it’s not already in the kernel?
- Later Steven was asked to add it
  - This became `make localmodconfig`
  - It sped up the build tremendously
  - It also hid the slow down caused by ftrace perl scripts!

text_poke()

Ftrace was the first to add runtime modification of code

```
<schedule>:
       0f 1f 44 00 00          nop
       53                      push   %rbx
       65 48 8b 1c 25 00 61    mov    %gs:0x16100,%rbx
       01 00
       ffffffff81a1491b: R_X86_64_32S  current_task
       48 8b 43 10             mov    0x10(%rbx),%rax
       48 85 c0                test   %rax,%rax
       74 10                   je     ffffffff81a14938 <schedule+0x28>
       f6 43 24 20             testb  $0x20,0x24(%rbx)
       75 49                   jne    ffffffff81a14977 <schedule+0x67>
       48 83 bb 20 0c 00 00    cmpq   $0x0,0xc20(%rbx)
       00
       74 1f                   je     ffffffff81a14957 <schedule+0x47>
       31 ff                   xor    %edi,%edi
       e8 a1 f8 ff ff          callq  ffffffff81a141e0 <__schedule>
```

text_poke()

Ftrace was the first to add runtime modification of code

```
<schedule>:
       <cc> 1f 44 00 00         <int3>nop
       53                       push   %rbx
       65 48 8b 1c 25 00 61     mov    %gs:0x16100,%rbx
       01 00
       ffffffff81a1491b: R_X86_64_32S  current_task
       48 8b 43 10              mov    0x10(%rbx),%rax
       48 85 c0                 test   %rax,%rax
       74 10                    je     ffffffff81a14938 <schedule+0x28>
       f6 43 24 20              testb  $0x20,0x24(%rbx)
       75 49                    jne    ffffffff81a14977 <schedule+0x67>
       48 83 bb 20 0c 00 00     cmpq   $0x0,0xc20(%rbx)
       00
       74 1f                    je     ffffffff81a14957 <schedule+0x47>
       31 ff                    xor    %edi,%edi
       e8 a1 f8 ff ff           callq  ffffffff81a141e0 <__schedule>
```

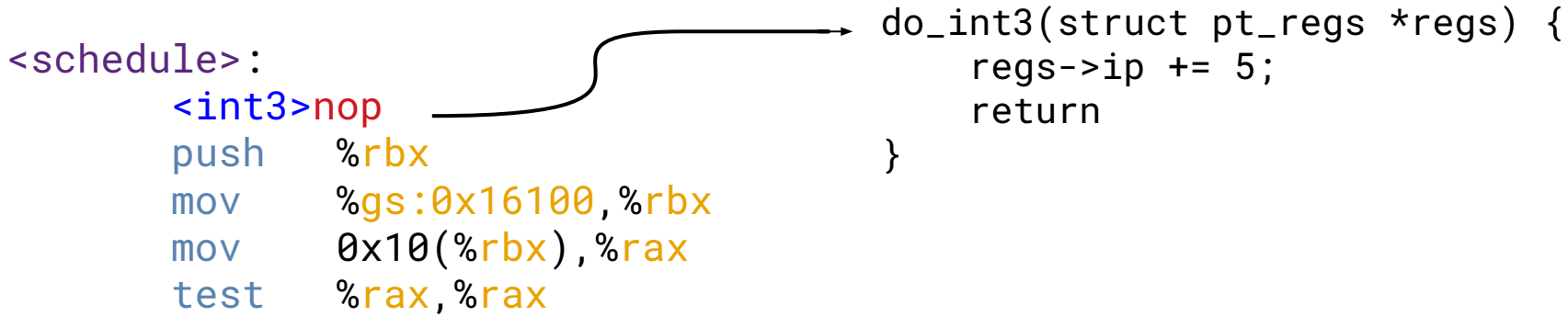
text_poke()

Ftrace was the first to add runtime modification of code

```
<schedule>:
        <int3>nop
        push   %rbx
        mov    %gs:0x16100,%rbx
        mov    0x10(%rbx),%rax
        test   %rax,%rax
```

text_poke()

Ftrace was the first to add runtime modification of code



text_poke()

Ftrace was the first to add runtime modification of code

```
<schedule>:
        <int3>nop
        push   %rbx
        mov    %gs:0x16100,%rbx
        mov    0x10(%rbx),%rax
        test   %rax,%rax
```



```
do_int3(struct pt_regs *regs) {
    regs->ip += 5;
    return
}
```

text_poke()

Ftrace was the first to add runtime modification of code

```
<schedule>:
        <int3>nop
        push   %rbx
        mov    %gs:0x16100,%rbx
        mov    0x10(%rbx),%rax
        test   %rax,%rax
```



```
do_int3(struct pt_regs *regs) {
    regs->ip += 5;
    return
}
```

text_poke()

Ftrace was the first to add runtime modification of code

```
<schedule>:
        <int3>nop
        push   %rbx
        mov    %gs:0x16100,%rbx
        mov    0x10(%rbx),%rax
        test   %rax,%rax
```

```
do_int3(struct pt_regs *regs) {
    regs->ip += 5;
    return
}
```

text_poke()

Ftrace was the first to add runtime modification of code

```
<schedule>:
       <cc> 1f 44 00 00         <int3>nop
       53                       push   %rbx
       65 48 8b 1c 25 00 61     mov    %gs:0x16100,%rbx
       01 00
       ffffffff81a1491b: R_X86_64_32S  current_task
       48 8b 43 10              mov    0x10(%rbx),%rax
       48 85 c0                 test   %rax,%rax
       74 10                    je     ffffffff81a14938 <schedule+0x28>
       f6 43 24 20              testb  $0x20,0x24(%rbx)
       75 49                    jne    ffffffff81a14977 <schedule+0x67>
       48 83 bb 20 0c 00 00     cmpq   $0x0,0xc20(%rbx)
       00
       74 1f                    je     ffffffff81a14957 <schedule+0x47>
       31 ff                    xor    %edi,%edi
       e8 a1 f8 ff ff           callq  ffffffff81a141e0 <__schedule>
```

text_poke()

Ftrace was the first to add runtime modification of code

```
<schedule>:
       <cc> 1b d0 1e 00         <int3>callq  ffffffff81c01930 <__fentry__>
       53                       push   %rbx
       65 48 8b 1c 25 00 61     mov    %gs:0x16100,%rbx
       01 00
       ffffffff81a1491b: R_X86_64_32S  current_task
       48 8b 43 10              mov    0x10(%rbx),%rax
       48 85 c0                 test   %rax,%rax
       74 10                    je     ffffffff81a14938 <schedule+0x28>
       f6 43 24 20              testb  $0x20,0x24(%rbx)
       75 49                    jne    ffffffff81a14977 <schedule+0x67>
       48 83 bb 20 0c 00 00     cmpq   $0x0,0xc20(%rbx)
       00
       74 1f                    je     ffffffff81a14957 <schedule+0x47>
       31 ff                    xor    %edi,%edi
       e8 a1 f8 ff ff           callq  ffffffff81a141e0 <__schedule>
```

text_poke()

Ftrace was the first to add runtime modification of code

```
<schedule>:
       e8 1b d0 1e 00          callq  ffffffff81c01930 <__fentry__>
       53                      push   %rbx
       65 48 8b 1c 25 00 61    mov    %gs:0x16100,%rbx
       01 00
       ffffffff81a1491b: R_X86_64_32S  current_task
       48 8b 43 10             mov    0x10(%rbx),%rax
       48 85 c0                test   %rax,%rax
       74 10                   je     ffffffff81a14938 <schedule+0x28>
       f6 43 24 20             testb  $0x20,0x24(%rbx)
       75 49                   jne    ffffffff81a14977 <schedule+0x67>
       48 83 bb 20 0c 00 00    cmpq   $0x0,0xc20(%rbx)
       00
       74 1f                   je     ffffffff81a14957 <schedule+0x47>
       31 ff                   xor    %edi,%edi
       e8 a1 f8 ff ff          callq  ffffffff81a141e0 <__schedule>
```

text_poke()

Static branches can use the same method!

```
if (static_key_false(&somekey)) {
    /* Do something unlikely */
}
```

text_poke()

Static branches can use the same method!

```
static int static_key_false(struct static_key *key)
{
        asm goto("1:"
                 ".byte 0xe9 \n\t .long 0\n\t /* nop */"
                 ".pushsection __jump_table,  \"a\" \n\t"
                  _ASM_PTR "1b, %l[" #label "], %c0 \n\t"
                  ".popsection \n\t"
                  : :  "i" (key) :  : l_yes);
         return false;
l_yes:
         return true;
}
```

text_poke()

Static branches can use the same method!

```
static int static_key_false(struct stat_key *key)
{
        asm goto("1:"
                 ".byte 0xe9 \n\t .long 0\n\t /* nop */"
                 ".pushsection __jump_table,  \"a\" \n\t"
                  _ASM_PTR "1b, %l[" #label "], %c0 \n\t"
                  ".popsection \n\t"
                  : :  "i" (key) :  : l_yes);
         return false;
l_yes:
         return true;
}
```



text_poke()

Static branches can use the same method!

```
static int static_key_false(struct static_key *key)
{
        asm goto("1:"
                 "jmp l_yes\n\t"
                 ".pushsection __jump_table,  \"a\" \n\t"
                  _ASM_PTR "1b, %l[" #label "], %c0 \n\t"
                  ".popsection \n\t"
                  : :  "i" (key) :  : l_yes);
         return false;
l_yes:
         return true;
}
```

text_poke()

Static branches can use the same method!

```
static int static_key_false(struct static_key *key)
{
        asm goto("1:"
                 "jmp l_yes\n\t"
                 ".pushsection __jump_table,  \"a\" \n\t"
                  _ASM_PTR "1b, %l[" #label "], %c0 \n\t"
                  ".popsection \n\t"
                  : :  "i" (key) :  : l_yes);
        return false;
l_yes:
        return true;
}
```



text_poke()

Static branches can use the same method!

```
if (static_key_false(&somekey)) {
    /* Do something unlikely */
}
```

text_poke()

Static branches can use the same method!

```
if (0) {
    /* Do something unlikely */
}
```

text_poke()

Static branches can use the same method!

```
if (1) {
    /* Do something unlikely */
}
```

text_poke()

- But you must enable CONFIG_JUMP_LABEL
  - General architecture-dependent options → Optimize very unlikely/likely branches

text_poke()

- But you must enable CONFIG_JUMP_LABEL
  - General architecture-dependent options → Optimize very unlikely/likely branches
- Without this, it's just a conditional branch

```
if (somekey->enabled) {

    /* Do something unlikely */

}
```

`cond_resched()`

- In `CONFIG_PREEMPT_NONE` the kernel does **not** preempt!

cond_resched()

- In CONFIG_PREEMPT_NONE the kernel does **not** preempt!
- If there is a long loop, the watchdog timer will trigger
  - Need to notify areas that can schedule

cond_resched()

- In CONFIG_PREEMPT_NONE the kernel does **not** preempt!
- If there is a long loop, the watchdog timer will trigger
  - Need to notify areas that can schedule
- Approximately 1455 cond_resch() in v6.11

cond_resched()

kernel/bpf/verifier.c:

```
for (i = 0; i < env->subprog_cnt; i++) {
    old_bpf_func = func[i]->bpf_func;
    tmp = bpf_int_jit_compile(func[i]);
    if (tmp != func[i] || func[i]->bpf_func != old_bpf_func) {
        verbose(env, "JIT doesn't support bpf-to-bpf calls\n");
        err = -ENOTSUPP;
        goto out_free;
    }
}
```

cond_resched()

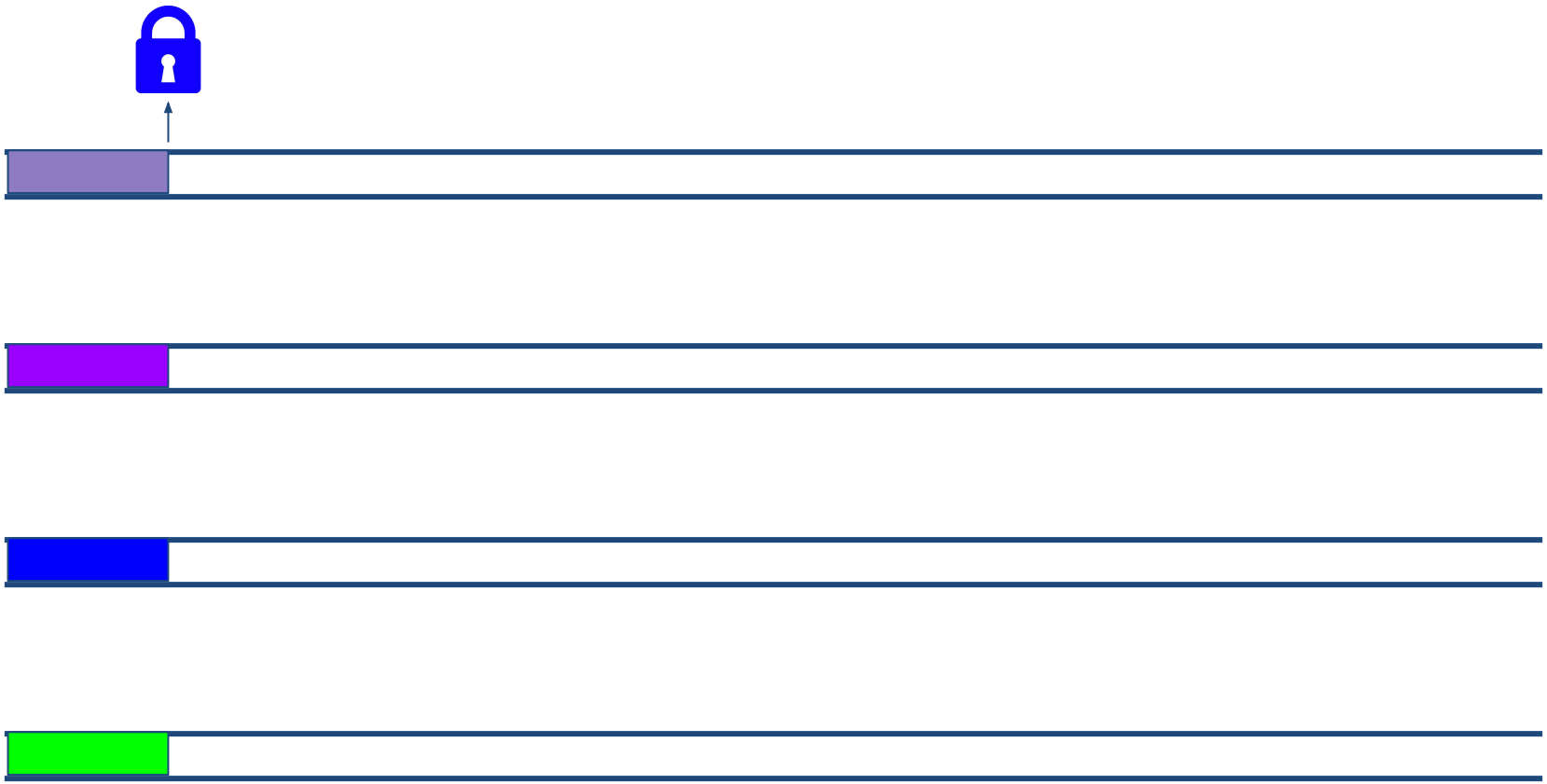
kernel/bpf/verifier.c:

```
  for (i = 0; i < env->subprog_cnt; i++) {
      old_bpf_func = func[i]->bpf_func;
      tmp = bpf_int_jit_compile(func[i]);
      if (tmp != func[i] || func[i]->bpf_func != old_bpf_func) {
          verbose(env, "JIT doesn't support bpf-to-bpf calls\n");
          err = -ENOTSUPP;
          goto out_free;
      }
+     cond_resched();
  }
```

What’s wrong with CONFIG_PREEMPT?

- It can preempt when locks are held
- Some workloads can take a big hit from it

CONFIG_PREEMPT_RT



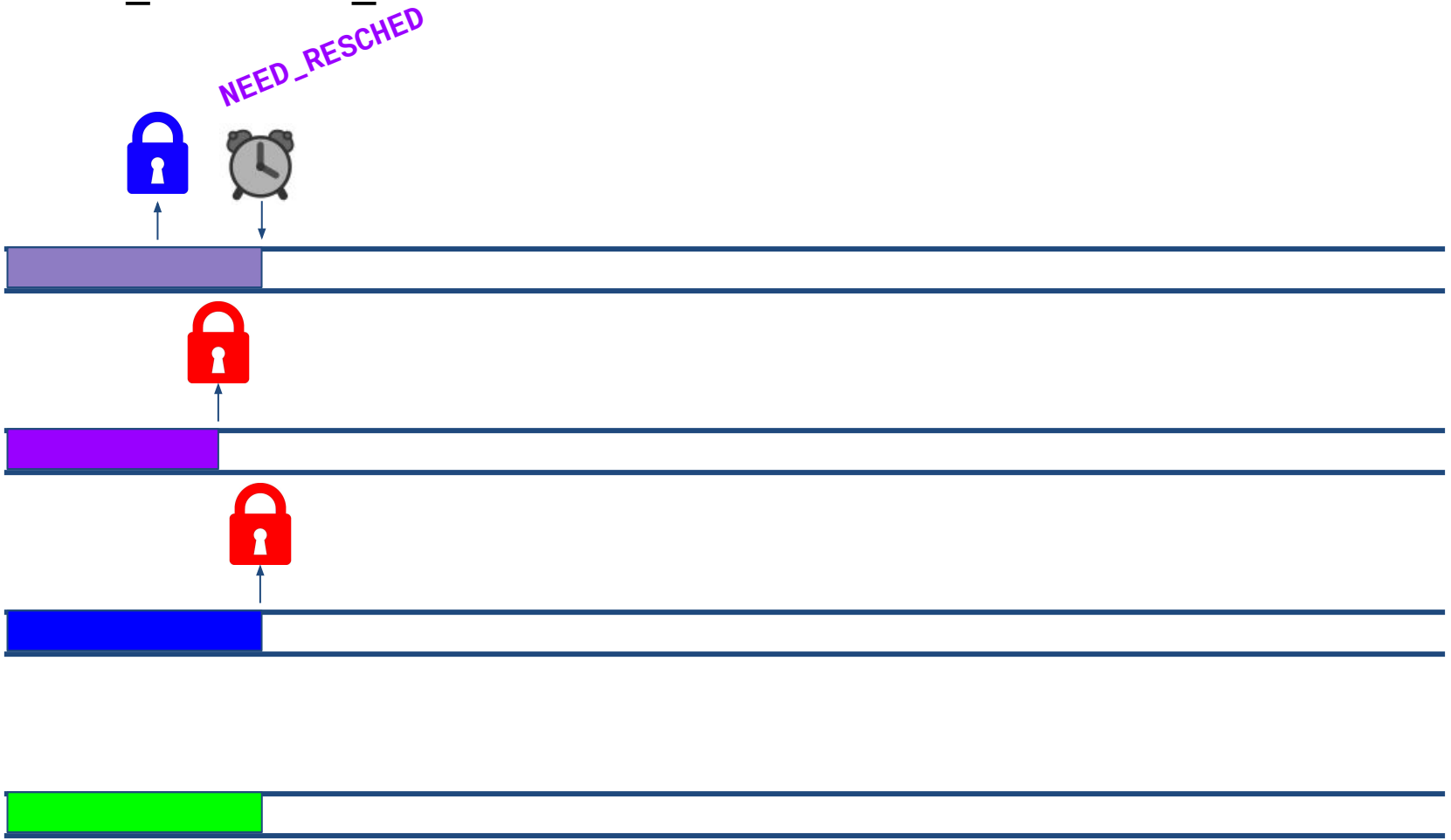
CONFIG_PREEMPT_RT



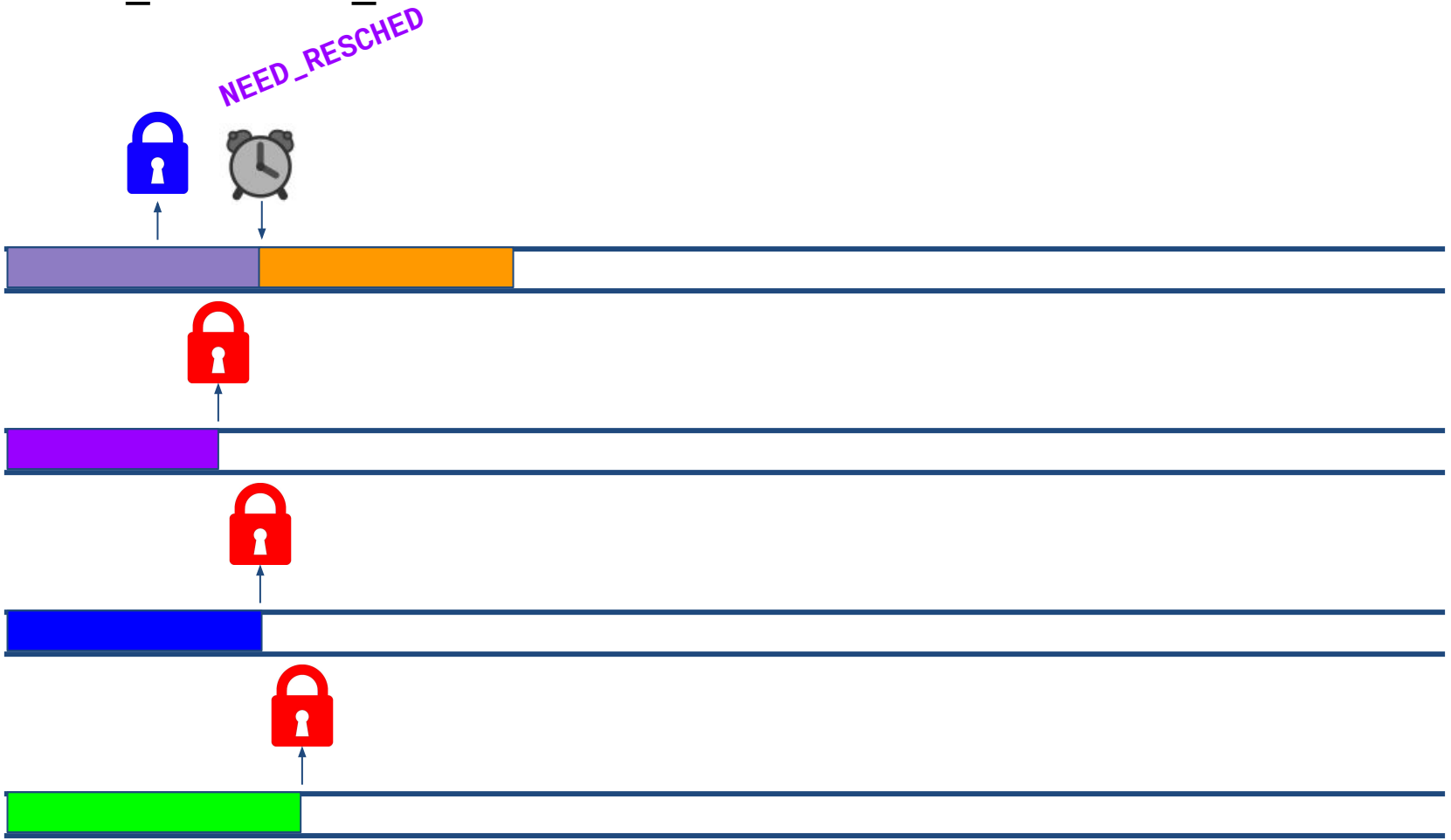
CONFIG_PREEMPT_RT



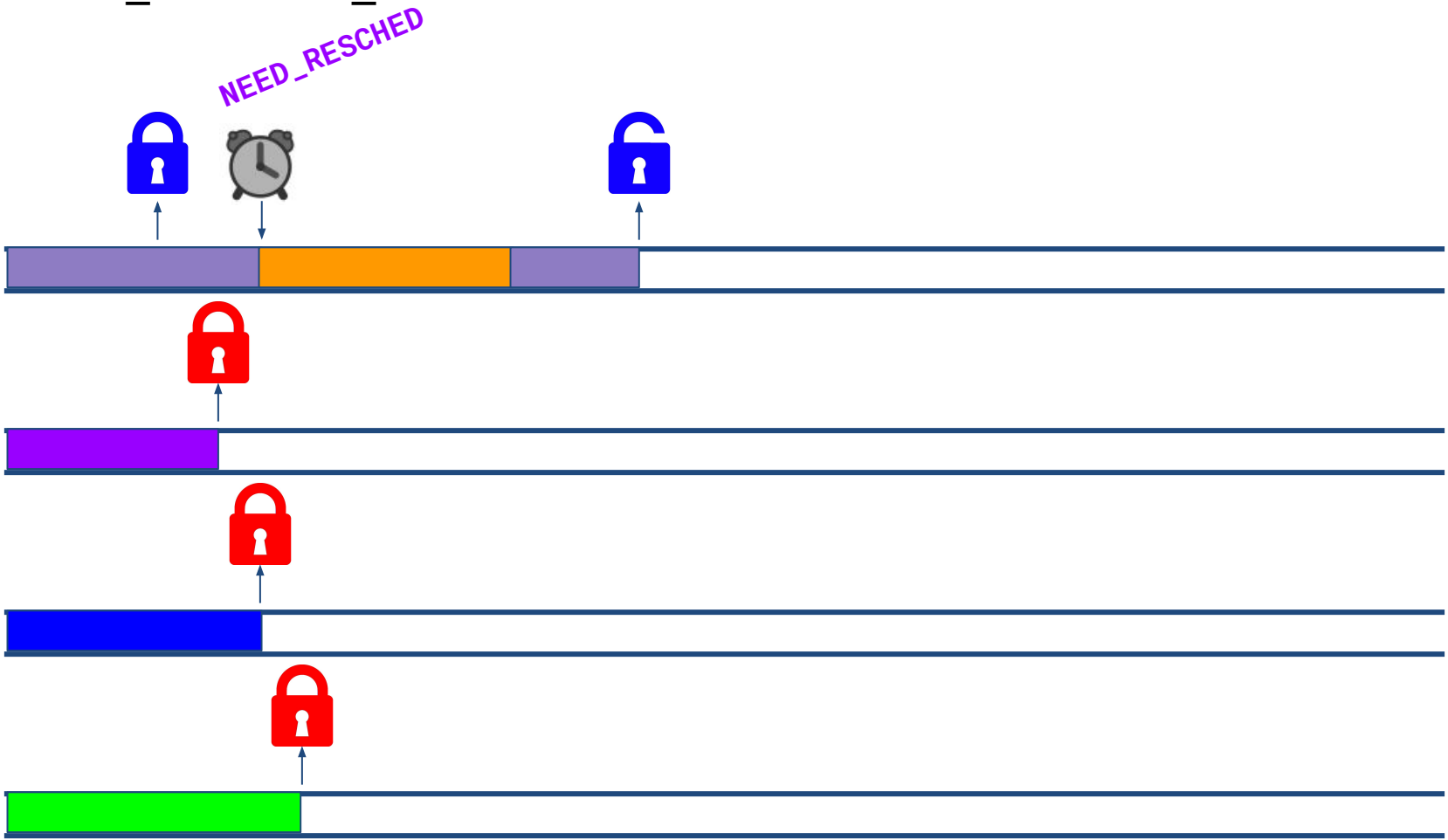
CONFIG_PREEMPT_RT



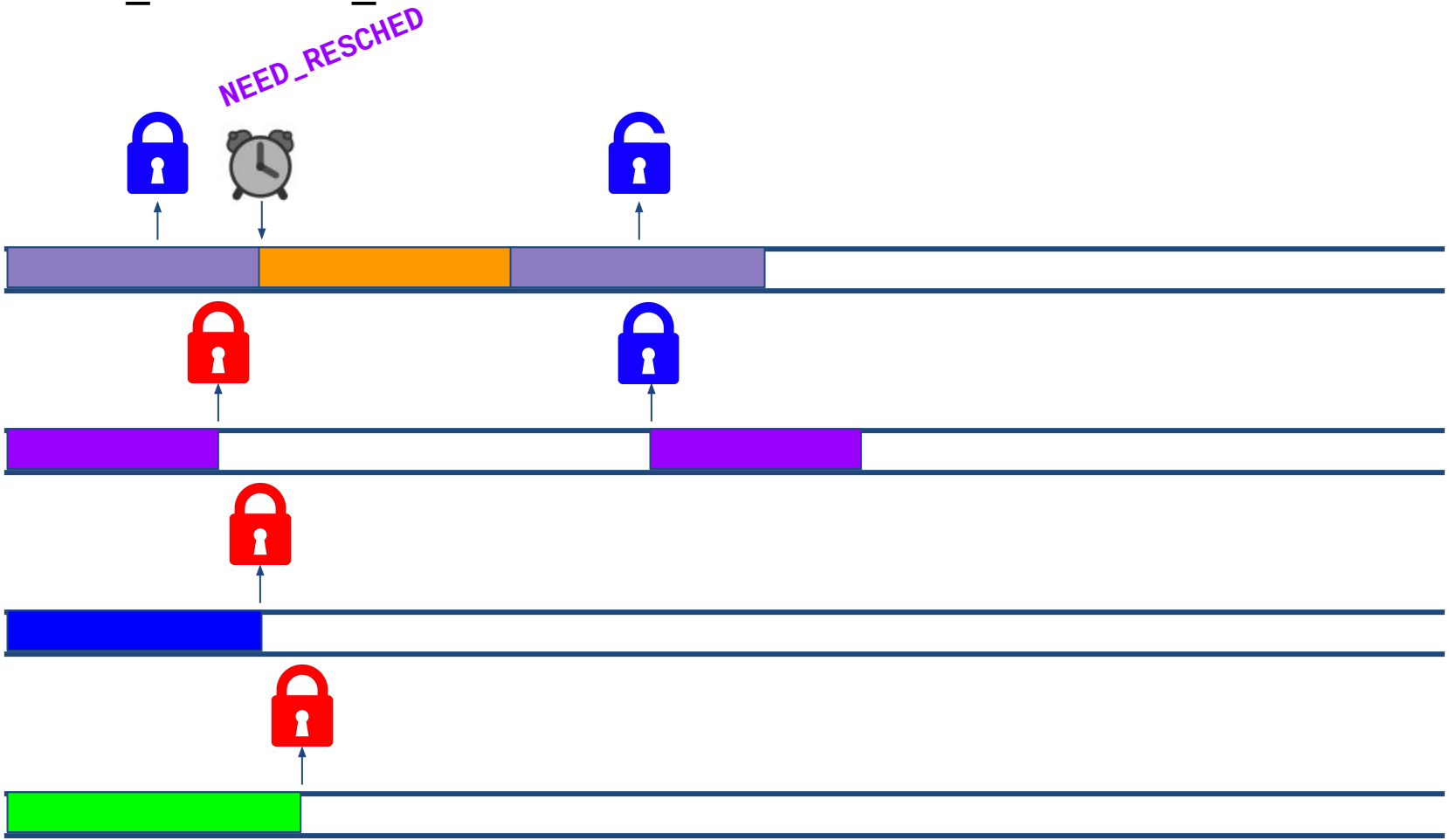
CONFIG_PREEMPT_RT



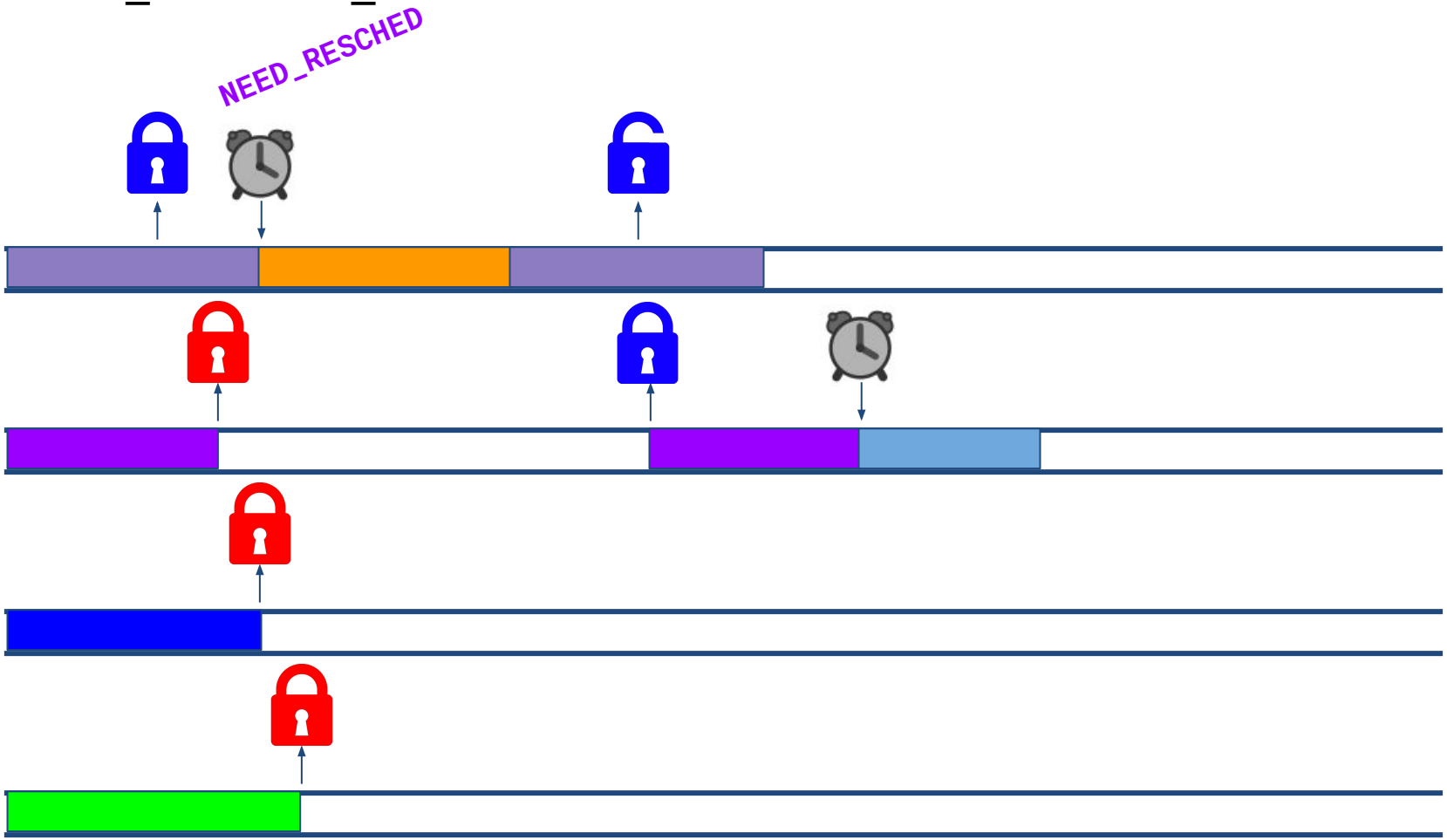
CONFIG_PREEMPT_RT



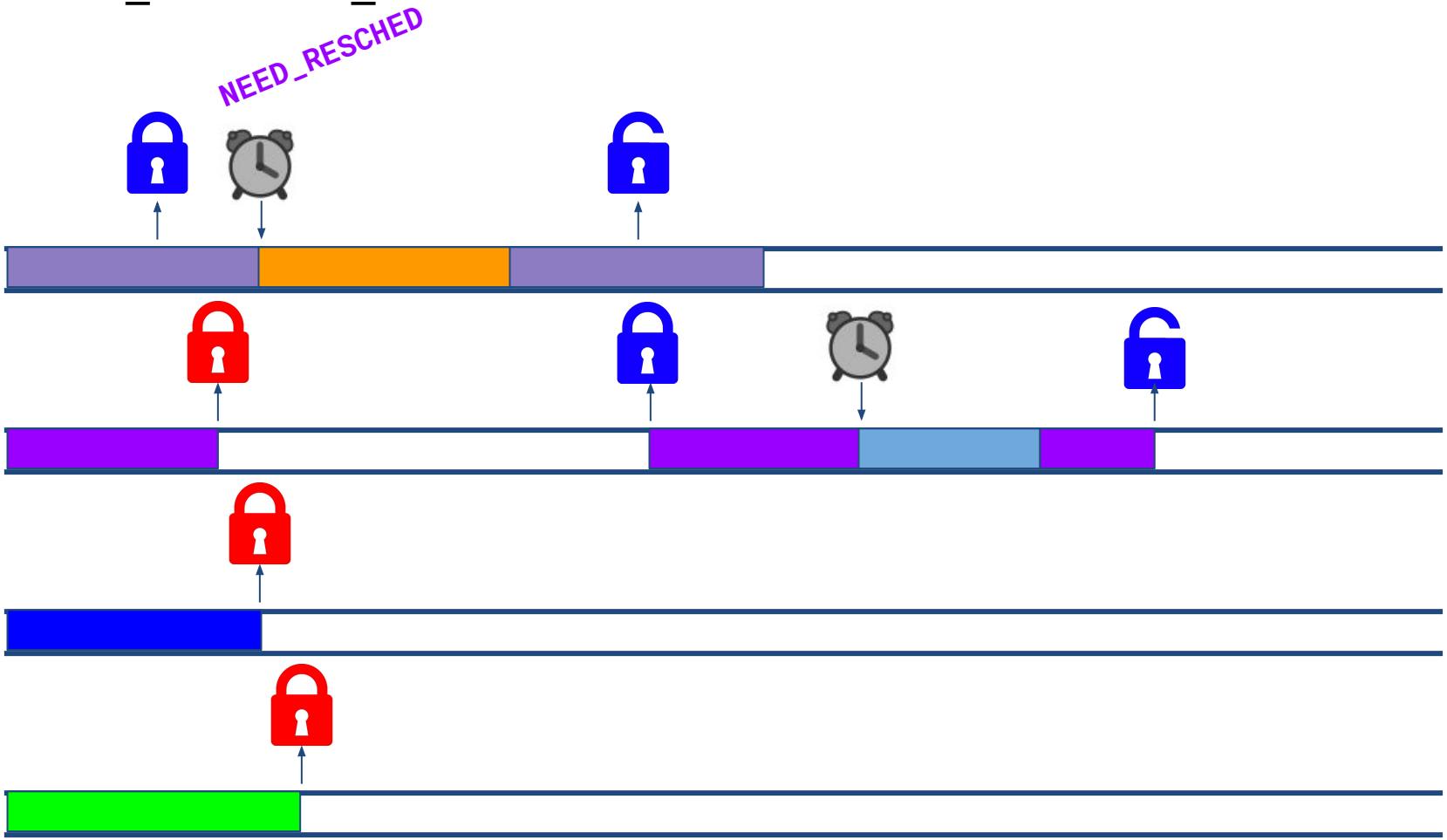
CONFIG_PREEMPT_RT



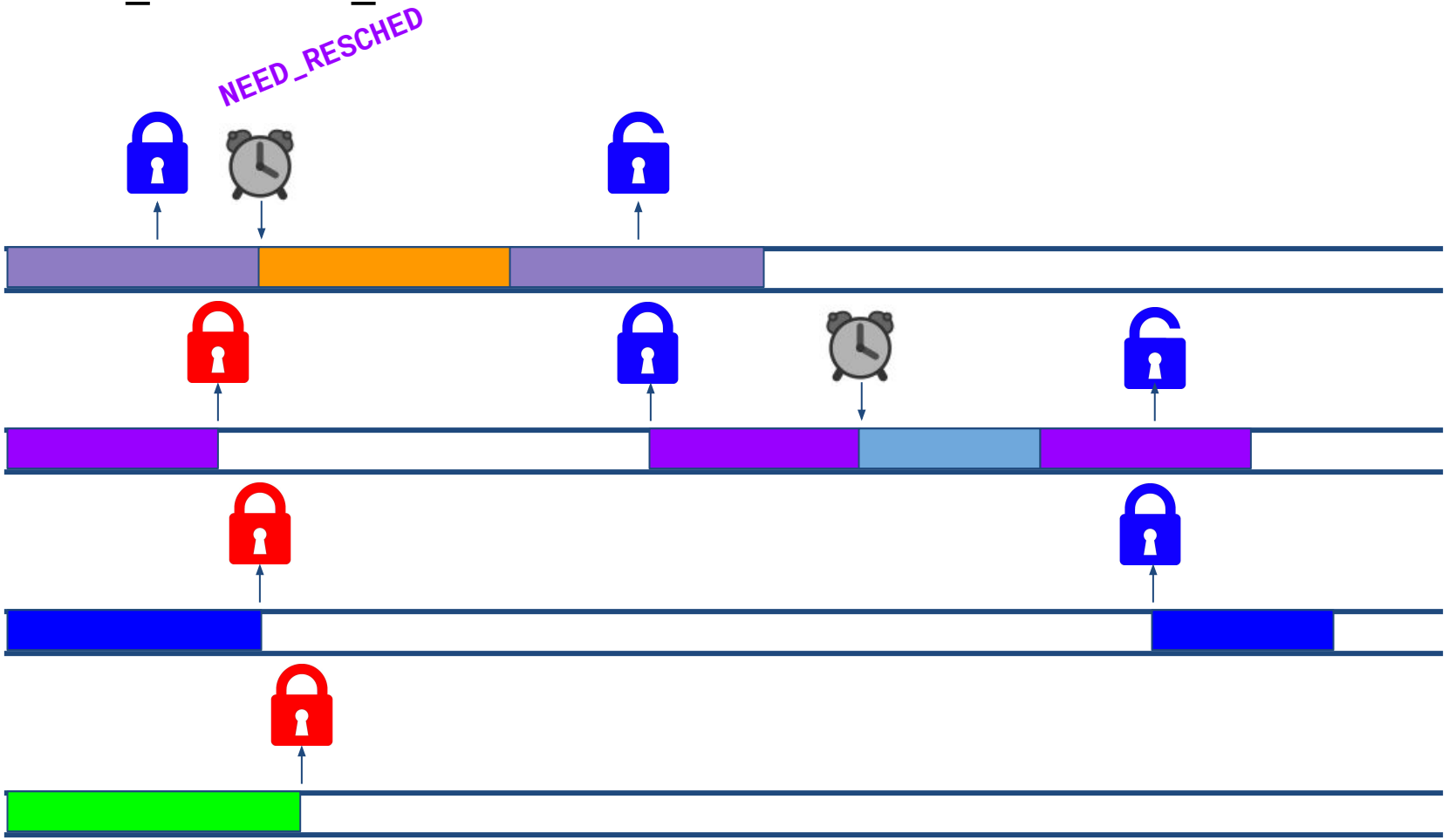
CONFIG_PREEMPT_RT



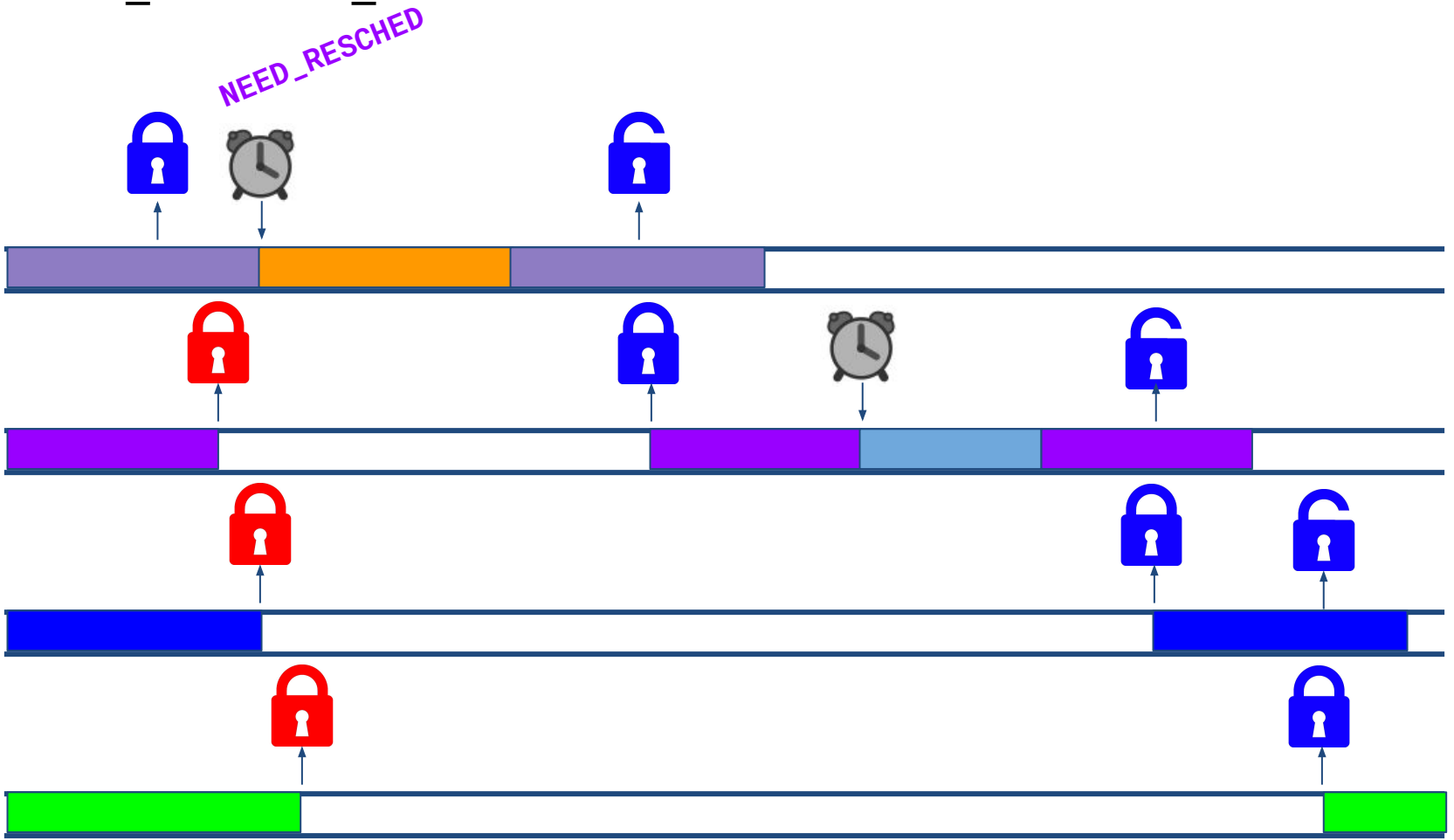
CONFIG_PREEMPT_RT



CONFIG_PREEMPT_RT

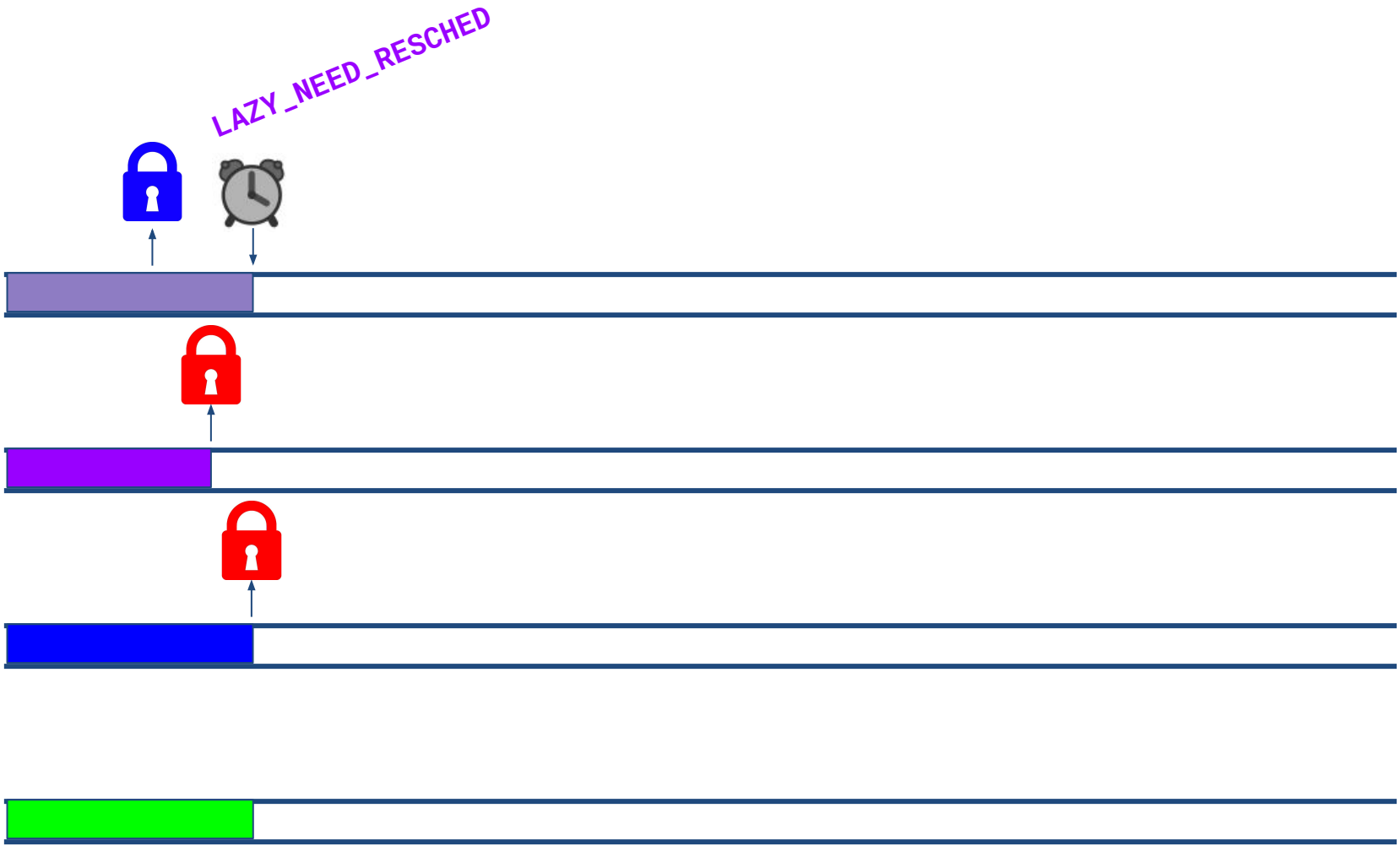


CONFIG_PREEMPT_RT



LAZY_NEED_RESCHED



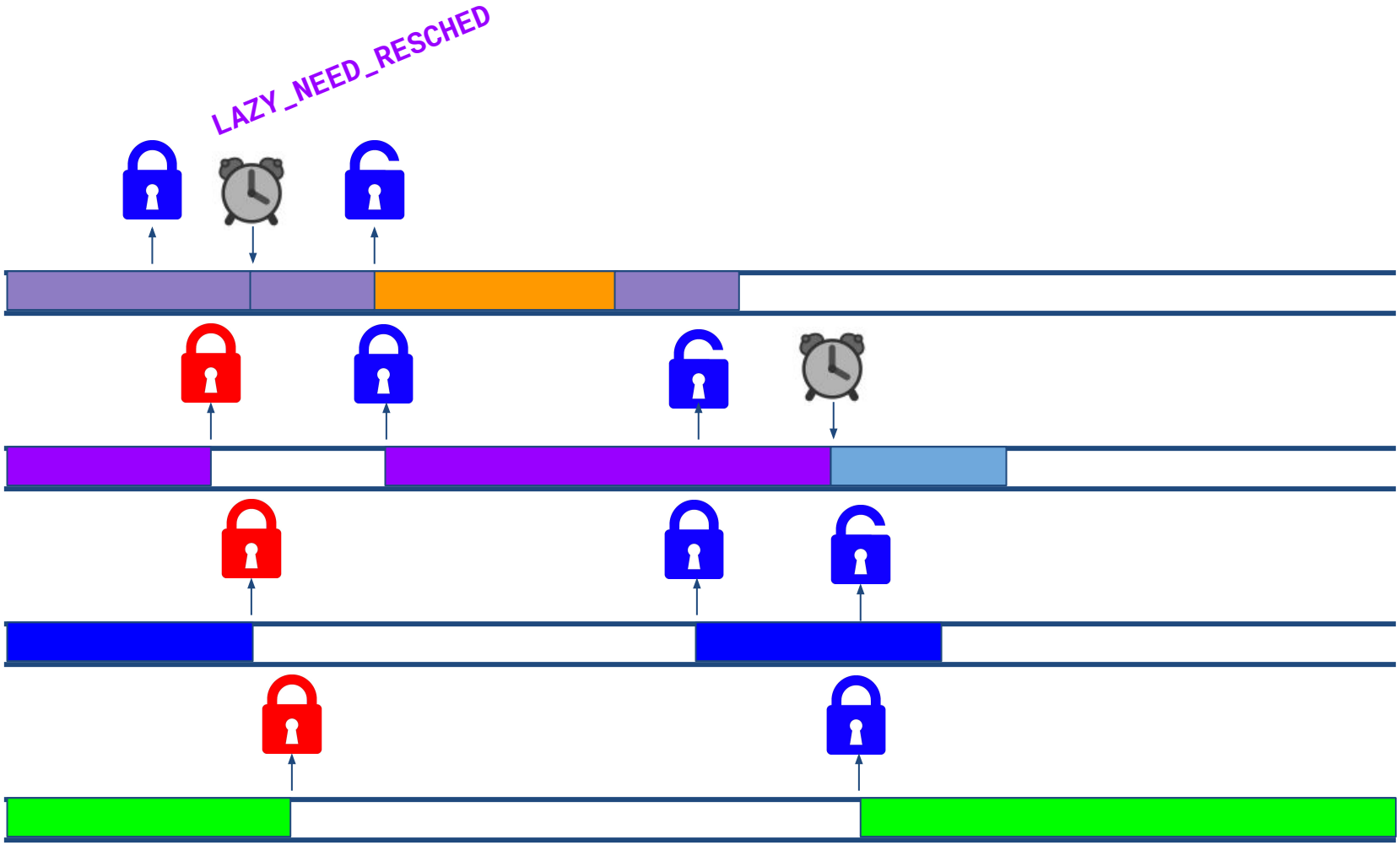

LAZY_NEED_RESCHED


LAZY_NEED_RESCHED


LAZY_NEED_RESCHED


LAZY_NEED_RESCHED


LAZY_NEED_RESCHED


LAZY_NEED_RESCHED



NEED_RESCHED



LAZY_NEED_RESCHED



CONFIG_PREEMPT_AUTO

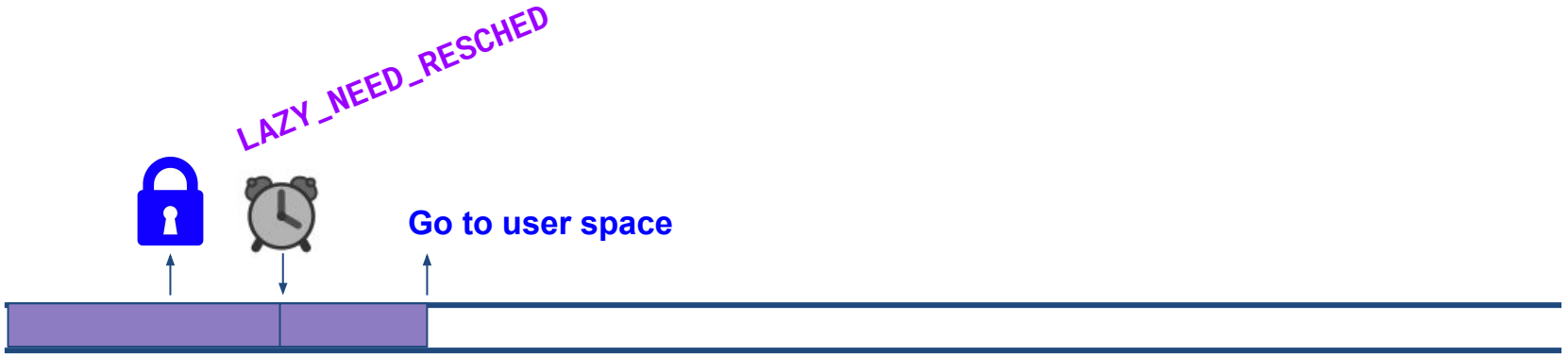
- Takes the lessons learned from LAZY_NEED_RESCHED of RT

Preempt Type	Schedule Tick	might_sleep()
PREEMPT_NONE	x	x
PREEMPT_VOLUNTARY	x	✅
PREEMPT	✅	✅

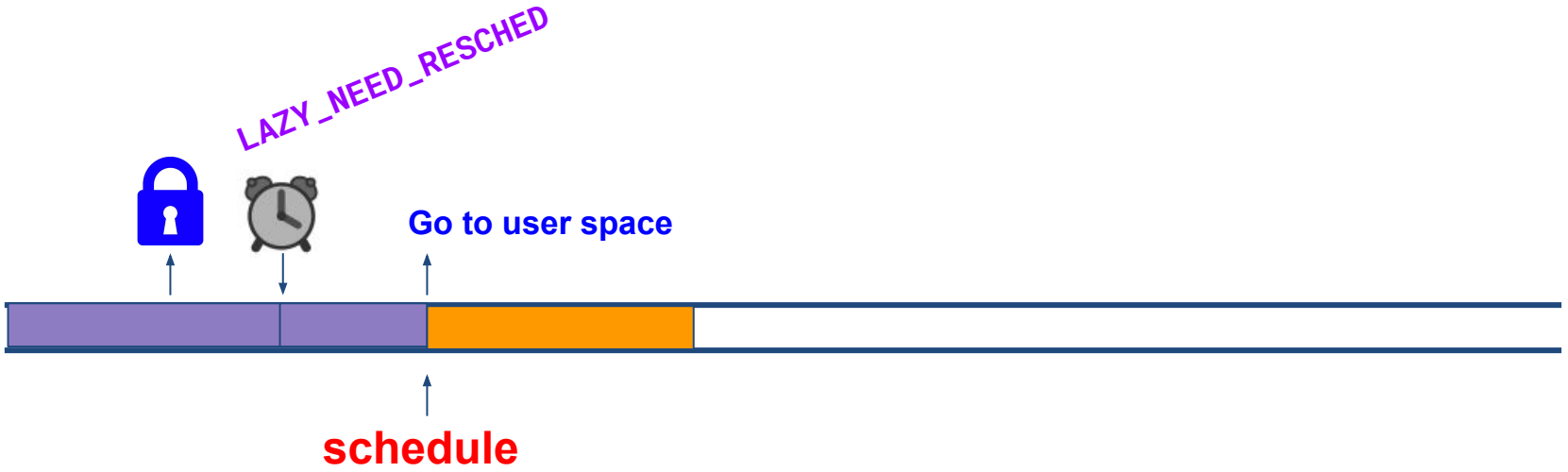
CONFIG_PREEMPT_AUTO



CONFIG_PREEMPT_AUTO



CONFIG_PREEMPT_AUTO



CONFIG_PREEMPT_AUTO



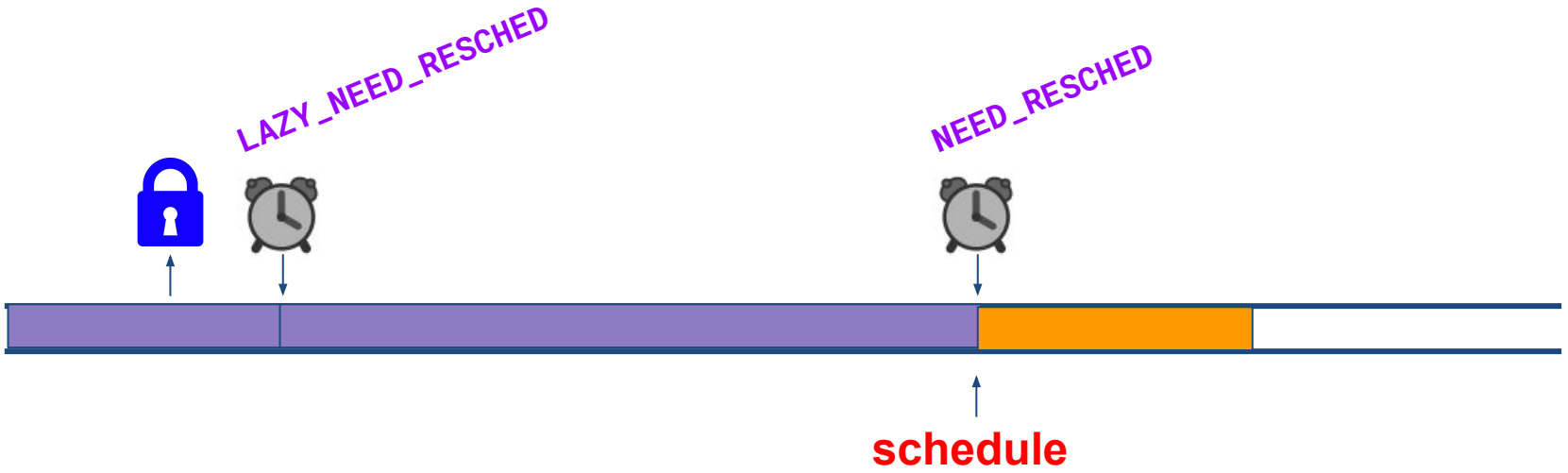
CONFIG_PREEMPT_AUTO



CONFIG_PREEMPT_AUTO



CONFIG_PREEMPT_AUTO



Questions?



Questions?

